

Домашняя работа (опциональная) по дисциплине «МОВС: Компьютерное зрение»

Разбор нейросетевого модуля трекинга

Выполнил:

Вольхин Данил Федорович

Email:

dfvolkhin@edu.hse.ru

Дата:

10 февраля 2025

😊 Первый раз в жизни делаю посадку на юпитер 😊

ANSI коды для цветов текста

```
In [1]: RED = "\033[31m"
GREEN = "\033[32m"
YELLOW = "\033[33m"
BLUE = "\033[34m"
RESET = "\033[0m" # сброс цветов до стандартных
```

Шаблоны markdown

Это комментарий в коде

```
def greet(name):
    print(f"Hello, {name}!")
```

code

Установка библиотек (Необязательно)

```
In [2]: # !pip install ultralytics pycocotools numpy scipy tqdm pillow scikit-learn albumen
```

Подготовка к написанию кода

Импорт библиотек

In [399...

```
print(f"{YELLOW}"+60*"-"+"{RESET}")
print(f"Библиотеки: \n")

# Дополнительные библиотеки
import platform # Узнать версию пайтона ;)
import os
import logging
import time
import sys
import random
import re
import shutil
from tqdm import tqdm
from functools import wraps
from typing import Any, Tuple, Union, Optional, List, Type, Callable, Dict
import collections
from collections import defaultdict
from collections import Counter
import zipfile
import rarfile
from dataclasses import dataclass
import tempfile

# Основные библиотеки
import IPython.display as ipd # Добавляет виджеты для ячеек юпитера
from IPython.display import HTML
from IPython import get_ipython
import ipykernel
print(f"python: {BLUE}{platform.python_version()}{RESET} ")
import matplotlib # Для рисунков
import matplotlib.pyplot as plt
print(f"matplotlib: {BLUE}{matplotlib.__version__}{RESET}")
import numpy as np # Для работы с массивами
print(f"numpy: {BLUE}{np.__version__}{RESET}")
import pandas as pd # Работа с таблицами
print(f"pandas: {BLUE}{pd.__version__}{RESET}")
import sklearn # Много полезного для ML
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
from sklearn.manifold import TSNE
print(f"sklearn: {BLUE}{sklearn.__version__}{RESET}")
import scipy
from scipy.spatial.distance import cdist, euclidean
from scipy.optimize import linear_sum_assignment
print(f"scipy: {BLUE}{scipy.__version__}{RESET}")
import pydantic # Для валидации данных
from pydantic import (BaseModel, Field, StrictStr, condecimal, StrictInt, StrictBool,
                      FilePath, DirectoryPath, ValidationError, root_validator, ConfigDict)
print(f"pydantic: {BLUE}{pydantic.__version__}{RESET}")
```

```

import fastapi
from fastapi import HTTPException, status
print(f"fastapi: {BLUE}{fastapi.__version__}{RESET}")
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.optim.lr_scheduler import ReduceLROnPlateau
from torch.utils.data import DataLoader, Subset, WeightedRandomSampler, Dataset
print(f"torch: {BLUE}{torch.__version__}{RESET}")
import torchvision
from torchvision import datasets
import torchvision.transforms as transforms
from torchvision.datasets import ImageFolder
from torchvision.transforms.functional import to_pil_image
import torchvision.models as models
print(f"torchvision: {BLUE}{torchvision.__version__}{RESET}")
import requests
print(f"requests: {BLUE}{requests.__version__}{RESET}")
import PIL
from PIL import Image, ImageOps
print(f"Pillow: {BLUE}{PIL.__version__}{RESET}")
import cv2
print(f"opencv: {BLUE}{cv2.__version__}{RESET}")
import ultralytics
from ultralytics import YOLO
print(f"ultralytics: {BLUE}{ultralytics.__version__}{RESET}")
import albumentations as A
print(f"albumentations: {BLUE}{A.__version__}{RESET}")
import pycocotools
import pycocotools.mask as maskUtils
from pytorch_metric_learning import losses

from rich.theme import Theme
from rich.logging import RichHandler
from rich.console import Console
from rich.pretty import install as pretty_install
from rich.traceback import install as traceback_install

print(f"{YELLOW}"+"60*"+"-"+f"{RESET}")

```

Библиотеки:

```
python: 3.9.0
matplotlib: 3.7.2
numpy: 1.23.0
pandas: 2.2.2
sklearn: 1.3.2
scipy: 1.13.1
pydantic: 2.10.6
fastapi: 0.115.7
torch: 1.12.1+cu116
torchvision: 0.13.1+cu116
requests: 2.32.3
Pillow: 10.4.0
opencv: 4.9.0
ultralitics: 8.3.78
albumentations: 2.0.4
```

Дополнительные настройки

```
In [4]: import warnings
warnings.filterwarnings("ignore")
import logging
# Отключение логирования для cmdstanpy
logging.getLogger('cmdstanpy').setLevel(logging.ERROR)
# Полезно при разработке собственных библиотек, юпитер будет переимпортировать модуль
%load_ext autoreload
%autoreload 1
```

Кастомный setup_logging

```
In [5]: log = None

def setup_logging(clean=False, debug=False):
    global log

    if log is not None:
        return log

    try:
        if clean and os.path.isfile('setup.log'):
            os.remove('setup.log')
        time.sleep(0.1) # prevent race condition
    except:
        pass

    if sys.version_info >= (3, 9):
        logging.basicConfig(level=logging.DEBUG, format='%(asctime)s | %(levelname)s | %(message)s',
                            filename='setup.log', filemode='a', encoding='utf-8', f
    else:
        logging.basicConfig(level=logging.DEBUG, format='%(asctime)s | %(levelname)s | %(message)s',
```

```

        filename='setup.log', filemode='a', force=True)

console = Console(log_time=True, log_time_format='%H:%M:%S-%f', theme=Theme({
    "traceback.border": "black",
    "traceback.border.syntax_error": "black",
    "inspect.value.border": "black",
}))
pretty_install(console=console)
traceback_install(console=console, extra_lines=1, width=console.width, word_wrap=True,
    suppress=[])
rh = RichHandler(show_time=True, omit_repeated_times=False, show_level=True, show_source=False,
    rich_tracebacks=True, log_time_format='%H:%M:%S-%f',
    level=logging.DEBUG if debug else logging.INFO, console=console)
rh.set_name(logging.DEBUG if debug else logging.INFO)
log = logging.getLogger("sd")
log.addHandler(rh)

return log

```

In [6]: `log = setup_logging()`

Валидация входящих данных для каждого класса

validate_with_pydantic

```

In [7]: def validate_with_pydantic(model_cls):
        """
        Декоратор для валидации данных с использованием Pydantic-модели.
        """

        def decorator(func):
            @wraps(func)
            def wrapper(*args, **kwargs):
                # Проверяем данные в аргументах функции
                try:
                    data = kwargs.get("entry", args[0] if args else None)
                    if not data:
                        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
                            detail="No data provided for validation.")

                    # Валидация данных
                    if isinstance(data, BaseModel):
                        data = data.dict(by_alias=True)
                    validated_data = model_cls(**data)
                    # Передаем валидированные данные дальше
                    kwargs["entry"] = validated_data
                    return func(*args, **kwargs)
                except ValidationError as ve:
                    log.exception("Validation failed", exc_info=ve)
                    raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
                        detail="Invalid data for Pydantic model.") from ve

            return wrapper

```

`return` decorator

auto_generate_docstring

```
In [8]: def auto_generate_docstring(cls: Type[BaseModel]) -> Type[BaseModel]:
        """
        Декоратор для автоматического добавления docstring в классы Pydantic.
        """

        def generate_docstring(model: Type[BaseModel]) -> str:
            """
            Генерация строки документации из описания полей модели Pydantic.
            """

            docstring = []
            for field_name, field_info in model.__fields__.items():
                field_details = f"Field '{field_name}':\n"
                if field_info.description: # Получение описания
                    field_details += f"Description: {field_info.description}\n"
                if field_info.examples: # Получение примеров
                    field_details += f"Examples: {field_info.examples}\n"
                docstring.append(field_details)
            return "\n".join(docstring)

        # Добавляем описание к существующему docstring
        cls.__doc__ = (cls.__doc__ or "") + "\n\n" + generate_docstring(cls)
        return cls
```

EntryGraduateModel

```
In [338... @auto_generate_docstring
class EntryGraduateModel(BaseModel):
    """
    Класс для валидации входных данных EntryGraduateModel
    """

    model_config = ConfigDict(arbitrary_types_allowed=True)
    Prefix: Optional[StrictStr] = Field("",
                                       alias="prefix",
                                       examples=["exp1"],
                                       description="Префикс к названию модели")

    Models: list[str] = Field(...,
                              alias="models",
                              examples=[["efficientnet_b0", "efficientnet_b1"]],
                              description="Список моделей для обучения")

    NameOptimizers: list[str] = Field(...,
                                      alias="name_optimizers",
                                      examples=["Adam"],
                                      description="Лист с названиями оптимизаторов")

    EmbSize: Optional[StrictInt] = Field(512,
                                         alias="emb_size",
                                         examples=[512],
                                         description="Размер эмбединга сиамской се")

    UseImageNetWeights: Optional[bool] = Field(False,
```

```

        alias="is_use_imagenet_weights",
        examples=[True],
        description="Использовать предобучен
SizeImg: Optional[tuple] = Field((64, 64),
        alias="size_img",
        examples=[(224, 224)],
        description="Размер изображения для обучения")
EmbSize: StrictInt = Field(1024,
        alias="emb_size",
        examples=[1024],
        description="Размер эмбединга модели")
PathData: Optional[StrictStr] = Field("./data",
        alias="path_to_data",
        examples=["./data"],
        description="Путь к директории с данными.
PathWeights: Optional[StrictStr] = Field("./weights",
        alias="path_to_weights",
        examples=["./weights"],
        description="Если сохраняем веса, то к
PathMetricsTrain: Optional[StrictStr] = Field("./metrics_train",
        alias="path_to_metrics_train",
        examples=["./metrics_train"],
        description="Куда сохраняем трени
PathMetricsTest: Optional[StrictStr] = Field("./metrics_test",
        alias="path_to_metrics_test",
        examples=["./metrics_test"],
        description="Куда сохраняем тестов
PathSavePlots: Optional[StrictStr] = Field("./plot",
        alias="path_to_save_plots",
        examples=["./plot"],
        description="Куда сохраняем графики
UseDevice: Optional[StrictStr] = Field(None,
        alias="use_device",
        examples=["cuda"],
        description="Какое устройство использова
StartLerningRate: Optional[condecimal(ge=0.00000001, le=0.01, decimal_places=8)

BatchSize: Optional[StrictInt] = Field(1,
        alias="batch_size",
        examples=[10],
        description="Размер пакета при обучении"
NumWorkers: Optional[StrictInt] = Field(0,
        alias="num_workers",
        examples=[0],
        description="Кол. используемых потоков
PinMemory: Optional[StrictBool] = Field(False,
        alias="pin_memory",
        examples=[False],
        description="Если True ускоряет загрузк
NumEpochs: Optional[StrictInt] = Field(3,
        alias="num_epochs",
        examples=[20],
        description="Количество эпох обучения дл
Seed: Optional[StrictInt] = Field(17,

```

```
alias="seed",
examples=[17],
description="Сажает зерно")
```

EntryInferenceModel

In [353...

```
@auto_generate_docstring
class EntryInferenceModel(BaseModel):
    """
    Класс для валидации входных данных EntryInferenceModel
    """
    model_config = ConfigDict(arbitrary_types_allowed=True)
    Prefix: Optional[StrictStr] = Field("",
                                       alias="prefix",
                                       examples=["exp1"],
                                       description="Префикс к названию модели")
    NameModel: StrictStr = Field(...,
                                 alias="name_model",
                                 examples=["efficientnet_b0_2024_10_02"],
                                 description="Название файла .pt")
    ImageList: list[StrictStr] = Field(...,
                                       alias="image_path_list",
                                       examples=[["./images/image.jpg"]],
                                       description="Список путей к изображениям")
    SizeImg: Optional[tuple] = Field((64, 64),
                                     alias="size_img",
                                     examples=[(224, 224)],
                                     description="Размер изображения для инференса")
    EmbSize: StrictInt = Field(1024,
                              alias="emb_size",
                              examples=[1024],
                              description="Размер эмбединга модели")
    PathWeights: Optional[StrictStr] = Field("./weights",
                                             alias="path_to_weights",
                                             examples=["./weights"],
                                             description="Путь к папке, где хранятся веса")
    UseDevice: Optional[StrictStr] = Field(None,
                                           alias="use_device",
                                           examples=["cuda"],
                                           description="Какое устройство использовать")
    Labels: Optional[bool] = Field(False,
                                   alias="labels",
                                   examples=["True"],
                                   description="Есть ли в названиях файлов метки? Да/Нет")
```

Загрузка и предобработка данных

Загрузка и распаковка набор данных PKU-Reid-Dataset


```

In [11]: class PKUReidDatasetDownloader:
    def __init__(self, download_url, target_dir='data'):
        self.download_url = download_url
        self.target_dir = target_dir
        self.dataset_zip = os.path.join(self.target_dir, 'PKU-Reid-Dataset.zip')
        os.makedirs(self.target_dir, exist_ok=True)

    def download_dataset(self):
        """Скачивание архива с датасетом"""
        response = requests.get(self.download_url, stream=True)
        total_size = int(response.headers.get('content-length', 0))

        with open(self.dataset_zip, 'wb') as file, tqdm(
            desc="Downloading PKU-Reid Dataset",
            total=total_size,
            unit='B',
            unit_scale=True,
            unit_divisor=1024
        ) as bar:
            for data in response.iter_content(chunk_size=1024):
                file.write(data)
                bar.update(len(data))

    def extract_dataset(self):
        """Распаковка архива"""
        with zipfile.ZipFile(self.dataset_zip, 'r') as zip_ref:
            zip_ref.extractall(self.target_dir)

    def organize_dataset(self):
        """Организация данных в структуру train/val/test"""
        source_folder = os.path.join(self.target_dir, 'PKUV1a_128x48')
        if not os.path.exists(source_folder):
            raise FileNotFoundError(f"Dataset folder {source_folder} not found!")

        # Создаем целевую структуру директорий
        splits = {
            'train': (1, 80),
            'val': (81, 97),
            'test': (98, 114)
        }

        for split, (start, end) in splits.items():
            split_dir = os.path.join(self.target_dir, split)
            os.makedirs(split_dir, exist_ok=True)

            # Фильтрация и перемещение файлов
            files = os.listdir(source_folder)
            for file in files:
                person_id = int(file.split('_')[0])
                if start <= person_id <= end:
                    src = os.path.join(source_folder, file)
                    dst = os.path.join(split_dir, file)
                    shutil.move(src, dst)

        # Удаление временных файлов

```




Формирование датасета для обучения

Класс SiameseDataset

In [276...

```
class SiameseDataset(Dataset):
    def __init__(self, root_dir: str, transform: Optional[callable] = None) -> None:
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []
        self.labels = []
        self.label_to_indices = defaultdict(list)
        self._prepare_dataset()

    def _prepare_dataset(self):
        for subdir, _, files in os.walk(self.root_dir):
            for file in files:
                if file.endswith('.png'):
                    label = int(file.split('.')[0].split('_')[0])
                    full_path = os.path.join(subdir, file)
                    self.image_paths.append(full_path)
                    self.labels.append(label)

        self.labels = np.array(self.labels)

        # Создаем словарь для быстрого доступа к индексам по меткам
        for idx, label in enumerate(self.labels):
            self.label_to_indices[label].append(idx)

    def __len__(self) -> int:
        return len(self.image_paths)

    def __getitem__(self, index: int) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        # Получаем первое изображение
        anchor_path = self.image_paths[index]
        anchor_label = self.labels[index]

        # Генерируем пару
        if random.random() < 0.5:
            # Положительная пара (одинаковый класс)
            same_class_indices = [i for i in self.label_to_indices[anchor_label] if i != index]
            if not same_class_indices:
                # Если нет других изображений в классе, создаем отрицательную пару
                positive = False
            else:
                pair_index = random.choice(same_class_indices)
                positive = True
        else:
            # Отрицательная пара (разный класс)
            different_label = random.choice([label for label in self.labels if label != anchor_label])
            different_indices = self.label_to_indices[different_label]
            pair_index = random.choice(different_indices)

        pair_path = self.image_paths[pair_index]
        pair_label = self.labels[pair_index]

        anchor_image = self.transform(torchvision.io.imread(anchor_path))
        pair_image = self.transform(torchvision.io.imread(pair_path))

        return anchor_image, pair_image, positive
```

```

        # Отрицательная пара (разные классы)
        positive = False

    if not positive:
        # Выбираем случайный другой класс
        other_labels = [l for l in self.label_to_indices.keys() if l != anchor_label]
        if not other_labels:
            # Если только один класс в датасете
            raise RuntimeError("Dataset contains only one class")
        chosen_label = random.choice(other_labels)
        pair_index = random.choice(self.label_to_indices[chosen_label])

    # Загружаем и преобразуем изображения
    anchor_img = Image.open(anchor_path).convert('RGB')
    pair_img = Image.open(self.image_paths[pair_index]).convert('RGB')

    if self.transform:
        anchor_img = self.transform(anchor_img)
        pair_img = self.transform(pair_img)

    return (
        anchor_img,
        pair_img,
        torch.tensor(1.0 if positive else 0.0, dtype=torch.float32)
    )

```

Класс SimpleDataset

In [277...

```

class SimpleDataset(Dataset):
    def __init__(self, root_dir: str, transform: Optional[callable] = None) -> None:
        """
        Инициализация датасета для валидации и тестирования.

        :param root_dir: корневая директория с изображениями
        :param transform: преобразования для изображений
        """
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []
        self.labels = []
        self._prepare_dataset()

    def _prepare_dataset(self) -> None:
        """Сбор данных и меток из файловой структуры."""
        for root, _, files in os.walk(self.root_dir):
            for file in files:
                if file.endswith('.png'):
                    file_path = os.path.join(root, file)
                    label = int(file.split('.')[0].split('_')[0])
                    self.image_paths.append(file_path)
                    self.labels.append(label)

    def __len__(self) -> int:
        """Возвращает общее количество изображений."""
        return len(self.image_paths)

```

```

def __getitem__(self, index: int) -> Tuple[torch.Tensor, torch.Tensor]:
    """Возвращает тензор изображения и соответствующую метку."""
    img_path = self.image_paths[index]
    label = self.labels[index]

    img = Image.open(img_path).convert('RGB')

    if self.transform:
        img = self.transform(img)

    return (
        img, # Тензор изображения [C, H, W]
        torch.tensor(label, dtype=torch.long) # Метка класса
    )

```

Класс ResizeWithPadding

In [278...

```

# Класс для приведения изображения к заданному размеру
class ResizeWithPadding:
    def __init__(self, size):
        self.size = size

    def __call__(self, img):
        # Приведение изображения к нужному размеру с сохранением соотношения сторон
        img.thumbnail((self.size, self.size), Image.Resampling.LANCZOS)

        # Добавление паддингов
        delta_width = self.size - img.size[0]
        delta_height = self.size - img.size[1]
        padding = (delta_width // 2, delta_height // 2, delta_width - (delta_width // 2), delta_height - (delta_height // 2))

        new_img = ImageOps.expand(img, padding, fill=(0, 0, 0))

        return new_img

```

Визуализация данных для обучения

In [287...

```

def visualize_augmentations(dataset, person_ids, n_samples=8):
    fig, axs = plt.subplots(len(person_ids), n_samples, figsize=(15, 5))

    for i, person_id in enumerate(person_ids):
        # Фильтруем изображения по ID
        indices = [idx for idx, label in enumerate(dataset.labels) if label == person_id]
        selected_indices = np.random.choice(indices, n_samples, replace=False)

        for j, idx in enumerate(selected_indices):
            img_tensor, label = dataset[idx]

            # Конвертируем тензор в PIL Image
            img = to_pil_image(img_tensor)

            axs[i, j].imshow(img)

```

```

        axs[i,j].axis('off')
        if j == 0:
            axs[i,j].set_title(f'ID: {person_id}')

plt.tight_layout()
plt.show()

```

```

In [296... def visualize_siamese_pairs(dataset, num_pairs=5):
    """
    Визуализация пар изображений из SiameseDataset.

    :param dataset: экземпляр SiameseDataset
    :param num_pairs: количество пар для отображения
    """
    fig, axs = plt.subplots(num_pairs, 2, figsize=(10, 2 * num_pairs))

    for i in range(num_pairs):
        # Получаем случайный индекс из датасета
        idx = np.random.randint(0, len(dataset))

        # Загружаем пару изображений и метку
        anchor_img, pair_img, label = dataset[idx]

        # Конвертируем тензоры в PIL Image
        anchor_img = to_pil_image(anchor_img)
        pair_img = to_pil_image(pair_img)

        # Отображаем изображения
        axs[i, 0].imshow(anchor_img)
        axs[i, 0].axis('off')
        axs[i, 0].set_title("Anchor Image")

        axs[i, 1].imshow(pair_img)
        axs[i, 1].axis('off')
        axs[i, 1].set_title(f"Pair Image (Label: {'Positive' if label == 1 else 'Negative'})")

    plt.tight_layout()
    plt.show()

```

```

In [300... # Преобразования для обучающего набора данных
train_transforms = transforms.Compose([
    ResizeWithPadding(128),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2,
        hue=0.1
    ),
    transforms.ToTensor()
])

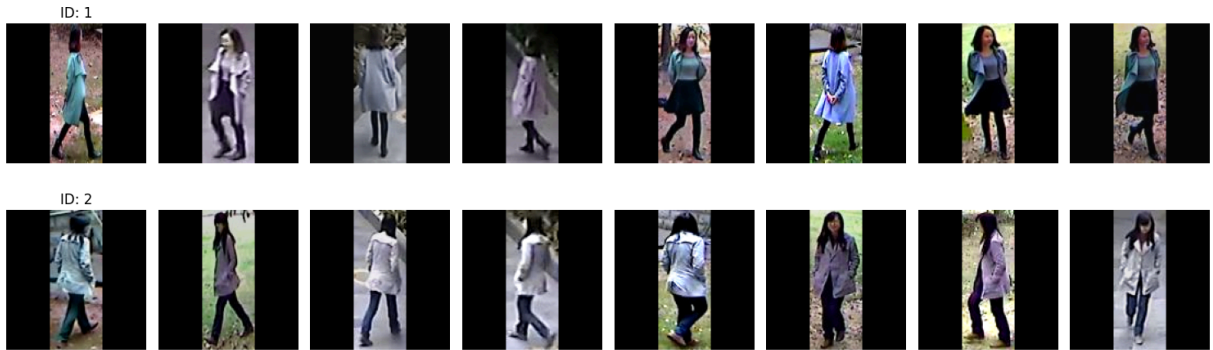
```

```

In [301... # Создаем временный SimpleDataset с тренировочными трансформациями
vis_dataset = SimpleDataset(root_dir='data/train', transform=train_transforms)

```

```
# Визуализируем первые два ID
visualize_augmentations(vis_dataset, person_ids=[1, 2])
```



```
In [302... # Создаем временный SiameseDataset с тренировочными трансформациями
vis_dataset = SiameseDataset(root_dir='data/train', transform=train_transforms)
# Визуализируем 5 случайных пар
visualize_siamese_pairs(vis_dataset, num_pairs=3)
```

Anchor Image



Pair Image (Label: Negative)



Anchor Image



Pair Image (Label: Positive)



Anchor Image

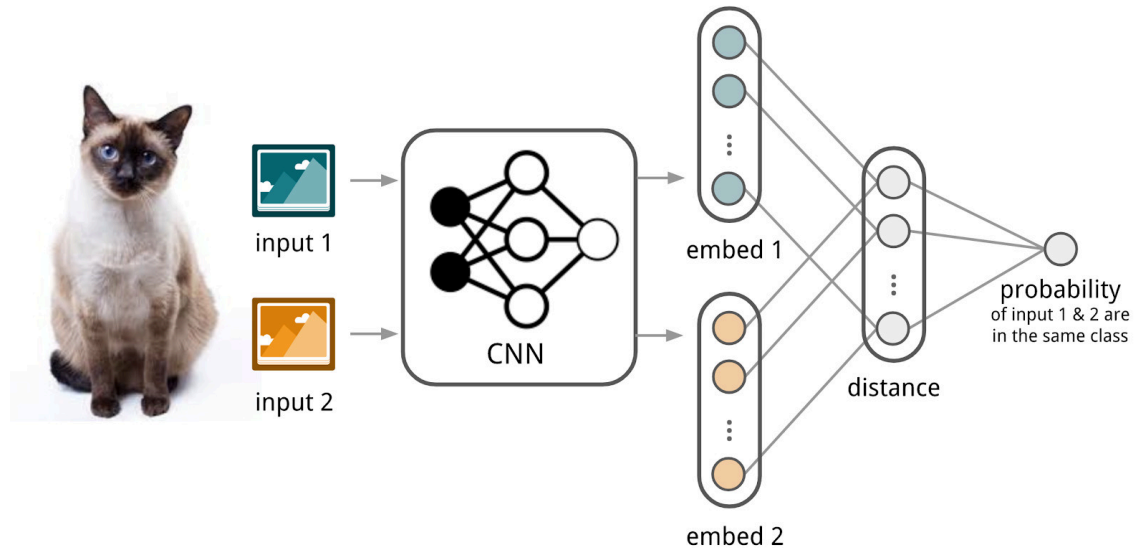


Pair Image (Label: Positive)



Сиамская нейронная сеть

Класс SiameseNetwork



In [303...

```
class SiameseNetwork(nn.Module):
    def __init__(
        self,
        name_model: str,
        emb_size: int = 512,
        imagenet: bool = True
    ):
        super().__init__()

        # Инициализируем параметры
        self.name_model = name_model
        self.emb_size = emb_size
        self.imagenet = imagenet

        # Получение списка доступных моделей
        self.model_list = sorted(name for name in models.__dict__
                                  if name.islower() and not name.startswith("__")
                                  and name != "get_weight"
                                  and callable(models.__dict__[name]))

        # Инициализируем архитектуру модели
        self.get_model()

        in_features = self.emb_size

        # Первая ветка полносвязных слоев
        self.fc = nn.Sequential(
            nn.Linear(in_features, self.emb_size * 2),
            nn.Tanh(),
            nn.Linear(self.emb_size * 2, self.emb_size * 2),
            nn.Tanh(),
```



```

        nn.Linear(self.emb_size * 2, self.emb_size * 2),
        nn.Tanh(),
        nn.Linear(self.emb_size * 2, self.emb_size),
    )

def get_model(self):
    # Проверка доступности модели в torchvision
    if self.name_model in self.model_list:
        # Проверка доступности весов ImageNet
        if self.imagenet:
            try:
                if self.name_model in ["inception_v3", "googlenet"]:
                    # Попытка загрузки модели с весами ImageNet
                    self.model = models.__dict__[self.name_model](init_weights="ImageNet")
                else:
                    if self.name_model in ["regnet_y_128gf", "vit_h_14"]:
                        self.model = models.__dict__[self.name_model](weights="ImageNet")
                    else:
                        self.model = models.__dict__[self.name_model](weights="ImageNet")
            except KeyError:
                # Если веса ImageNet для данной модели недоступны, загрузка без весов
                print(f"Предобученные веса ImageNet для модели {self.name_model} недоступны. Загрузка без весов.")
                if self.name_model in ["inception_v3", "googlenet"]:
                    # Попытка загрузки модели с весами ImageNet
                    self.model = models.__dict__[self.name_model](init_weights="ImageNet")
                else:
                    self.model = models.__dict__[self.name_model](weights=None)
        else:
            # Если не указано использование весов ImageNet, загрузить модель без весов
            if self.name_model in ["inception_v3", "googlenet"]:
                self.model = models.__dict__[self.name_model](init_weights=False)
            else:
                self.model = models.__dict__[self.name_model](weights=None)

    try:
        name_without_numbers = re.sub(r'\d+', '', self.name_model)
        # Проверка доступности слоя classifier
        if hasattr(self.model, 'classifier'):
            if name_without_numbers == "densenet":
                num_features = self.model.classifier.in_features
                self.model.classifier = nn.Linear(num_features, self.emb_size)
            elif name_without_numbers == "squeeze":
                num_features = 512
                self.model.classifier[-1] = nn.Linear(num_features, self.emb_size)
            else:
                num_features = self.model.classifier[-1].in_features
                self.model.classifier[-1] = nn.Linear(num_features, self.emb_size)
        elif hasattr(self.model, 'last_linear'):
            num_features = self.model.last_linear.in_features
            self.model.last_linear = nn.Linear(num_features, self.emb_size)
        elif hasattr(self.model, 'fc'):
            num_features = self.model.fc.in_features
            self.model.fc = nn.Linear(num_features, self.emb_size)
        elif hasattr(self.model, 'head'):

```

```

        num_features = self.model.head.in_features
        self.model.head = nn.Linear(num_features, self.emb_size)
    elif hasattr(self.model, "heads"):
        num_features = self.model.heads[0].in_features
        self.model.heads[0] = nn.Linear(num_features, self.emb_size)
    else:
        print(f"Слой classifier не найден в модели. name_model:{self.name_model}")
except Exception as ex:
    print(f"Exception: {ex}, name_model: {self.name_model}")
else:
    print(f"Модель {self.name_model} не найдена в torchvision."
          f"\nСписок доступных моделей: {self.model_list}")

def forward_once(self, x: torch.Tensor) -> torch.Tensor:
    """
    Обработка одного изображения через выбранную ветку.

    :param x: входное изображение
    :param branch: номер ветки (1 или 2)
    :return: эмбединг изображения
    """
    features = self.model(x)
    return self.fc(features)

def forward(self, input1: torch.Tensor, input2: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """
    Обработка пары изображений через две ветки.

    :param input1: первое изображение
    :param input2: второе изображение
    :return: два эмбединга (от первой и второй ветки)
    """
    # Первое изображение обрабатывается через первую ветку
    embedding1 = self.forward_once(input1)

    # Второе изображение обрабатывается через вторую ветку
    embedding2 = self.forward_once(input2)

    return embedding1, embedding2

```

Контрастивная функция потерь

Класс ContrastiveLoss

$$L = y * ||f(I_p) - f(I_q)||_2^2 + (1 - y) * \max\{0, m - ||f(I_p) - f(I_q)||_2\}^2$$

$y = 1$ if images I_p, I_q are similar (positive pairs)

$y = 0$ if images I_p, I_q are dissimilar (negative pairs)

m – margin which dissimilar samples should stay apart from

$$L = y * ||f(I_p) - f(I_q)||_2^2 + (1 - y) * \max(0, m - ||f(I_p) - f(I_q)||_2)^2$$

In [304...

```
class ContrastiveLoss(nn.Module):
    def __init__(self, margin: float = 1.0):
        super().__init__()
        self.margin = margin

    def forward(self, output1: torch.Tensor, output2: torch.Tensor, label: torch.Tensor):
        # Вычисляем Евклидово расстояние между эмбедами
        euclidean_distance = F.pairwise_distance(output1, output2)

        # Вычисляем потери для похожих и разных пар
        loss_similar = label * torch.pow(euclidean_distance, 2)
        loss_dissimilar = (1 - label) * torch.pow(
            torch.clamp(self.margin - euclidean_distance, min=0.0),
            2
        )

        # Усредняем потери по батчу
        loss_contrastive = torch.mean(loss_similar + loss_dissimilar)
        return loss_contrastive
```

Функция для расчета метрики mAP

In [305...

```
def calculate_map(all_embs: List[np.ndarray], all_labels: List[int]) -> float:
    """
    Вычисление среднего значения AP (mAP) для всех эмбеддингов и их меток.

    :param all_embs: список эмбеддингов изображений.
    :param all_labels: список меток (id), соответствующих эмбеддингам.
    :return: mAP.
    """
    aps = []

    for idx in range(len(all_labels)):
        # Создаем временные копии данных
        embs = all_embs[idx].copy()
        labels = all_labels[idx].copy()

        # Извлекаем якорь и удаляем его из списков
        anchor_emb = embs[0]
        anchor_label = labels[0]

        # Преобразуем в numpy массивы
        embs_array = np.array(embs)
        anchor_array = np.array(anchor_emb)

        # Вычисляем евклидовы расстояния
        distances = np.linalg.norm(embs_array - anchor_array, axis=1)

        # Сортируем метки по расстояниям
        sorted_indices = np.argsort(distances)
        sorted_labels = np.array(labels)[sorted_indices]
```

```

# Вычисляем AP для текущего якоря
correct = (sorted_labels == anchor_label)
relevant = np.where(correct)[0]

if len(relevant) == 0:
    ap = 0.0
else:
    cumulative_correct = np.cumsum(correct)
    precision_at_k = cumulative_correct / (np.arange(len(correct)) + 1)
    ap = np.sum(precision_at_k[correct]) / len(relevant)

aps.append(ap)

return np.mean(aps)

```

Обучение моделей

GraduateModel

In [306...

```

class GraduateModel:

    def __init__(self,
                 prefix: str = "",
                 name_model: str = "efficientnet_b0",
                 path_to_data: str = "./data",
                 path_to_weights: str = "./weights",
                 path_to_metrics_train: str = "./metrics_train",
                 path_to_metrics_test: str = "./metrics_test",
                 is_use_imagenet_weights: bool = True,
                 name_optimizer: str = "Adam",
                 emb_size: int = 512,
                 num_epochs: int = 3,
                 batch_size: int = 1,
                 train_size_img: tuple = (128, 128),
                 use_device: str = None,
                 start_learning_rate: float = None,
                 pin_memory: bool = False,
                 num_workers: int = 0,
                 seed: int = 17):

        self.name_model = name_model
        self.name_model_user = f"{self.name_model}_{prefix}" if prefix != "" else self.name_model
        self.path_to_data = path_to_data
        directories_to_create = [path_to_weights, path_to_metrics_train, path_to_metrics_test]
        self.create_directories_if_not_exist(*directories_to_create)
        self.path_to_weights = os.path.join(path_to_weights, f"{self.name_model_user}")
        self.path_to_metrics_train = os.path.join(path_to_metrics_train, f"{self.name_model_user}")
        self.path_to_metrics_test = os.path.join(path_to_metrics_test, f"{self.name_model_user}")
        self.num_epochs = num_epochs
        self.batch_size = batch_size
        self.size_img = train_size_img

```

```

self.name_optimizer = name_optimizer
self.emb_size = emb_size
self.model = None
self.train_loader = None
self.valid_loader = None
self.test_loader = None
self.criterion = None
self.scheduler = None
self.imagenet = is_use_imagenet_weights
self.use_device = use_device
self.start_learning_rate = start_learning_rate
self.pin_memory = pin_memory
self.num_workers = num_workers
self.seed = seed

# Получение списка доступных моделей
self.model_list = sorted(name for name in models.__dict__
                          if name.islower() and not name.startswith("__")
                          and name != "get_weight"
                          and callable(models.__dict__[name]))

# Перемещение модели на GPU, если CUDA доступен
if self.use_device == None:
    self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
elif self.use_device == "cuda":
    self.device = torch.device("cuda")
elif self.use_device == "cpu":
    if self.pin_memory == True:
        raise Exception("pin_memory musn't be True with use_device cpu")
    self.device = torch.device("cpu")
else:
    raise Exception("use_device must be cuda or cpu or None")

@staticmethod
def seed_everything(seed):
    # Задаём seed для генераторов псевдослучайных чисел в Python, PyTorch и NumPy
    random.seed(seed)
    torch.manual_seed(seed)
    np.random.seed(seed)
    # Если доступна CUDA, задаём seed для GPU
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    # Устанавливаем поведение CuDNN в детерминированный режим
    torch.backends.cudnn.deterministic = True
    # Включаем режим оптимизации CuDNN
    torch.backends.cudnn.benchmark = True
    # Создаем генератор случайных чисел PyTorch и задаём ему seed
    g = torch.Generator()
    g.manual_seed(seed)

# Функция для установки seed в DataLoader
@staticmethod
def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % 2**32
    np.random.seed(worker_seed)

```

```

random.seed(worker_seed)

def graduate(self):
    # Фиксируем зерно
    self.seed_everything(self.seed)
    # Получаем трансформер
    self.get_transform()
    # Получаем генераторы обучения, валидации и теста
    self.get_loaders()
    # Загружаем модель
    self.get_model()
    # Определяем оптимизатор, функцию потерь и планировщик
    self.get_opt_crit_sh()
    # Выводим информацию
    print(self.__str__())
    # Обучаем
    self.train_model()
    # Тестируем
    self.evaluate_model()

def __str__(self):

    return f"""

    -----

    Выбранная модель: {self.name_model}
    Пользовательское название модели: {self.name_model_user}
    Выбранный оптимизатор: {self.name_optimizer}

    -----

    """

def create_directories_if_not_exist(self, *directories):
    for directory in directories:
        if not os.path.exists(directory):
            os.makedirs(directory)

# Функция для сохранения модели
def save_model(self):
    # Сохраняем модель
    torch.save({
        'epoch': self.num_epochs,
        'model_state_dict': self.model.state_dict(),
        'optimizer_state_dict': self.optimizer.state_dict(),
        f'{self.path_to_weights}"')

def save_metrics_train(self,
                       train_loss_values,
                       score_values):
    metrics = {
        'train_loss': train_loss_values,
        'score': score_values
    }
    # Сохранение метрик

```

```

torch.save(metrics, self.path_to_metrics_train)

def save_metrics_test(self,
                      score):
    metric = {
        'score': score
    }
    # Сохранение метрик
    torch.save(metric, self.path_to_metrics_test)

def get_transform(self):
    self.transform = {
        "train": transforms.Compose([
            ResizeWithPadding(self.size_img[0]),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.ColorJitter(
                brightness=0.2,
                contrast=0.2,
                saturation=0.2,
                hue=0.1
            ),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406], # Средние значения для ImageNet
                std=[0.229, 0.224, 0.225]   # Стандартные отклонения для ImageNet
            )
        ]),
        "valid": transforms.Compose([
            ResizeWithPadding(self.size_img[0]),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406], # Средние значения для ImageNet
                std=[0.229, 0.224, 0.225]   # Стандартные отклонения для ImageNet
            )
        ])
    }

# Функция для загрузки данных
def get_loaders(self):
    # Формирование датасетов
    self.train_dataset = SiameseDataset(root_dir='data/train', transform=self.transform)
    self.valid_dataset = SimpleDataset(root_dir='data/val', transform=self.transform)
    self.test_dataset = SimpleDataset(root_dir='data/test', transform=self.transform)
    # Формирование загрузчиков данных
    self.train_loader = DataLoader(self.train_dataset, batch_size=self.batch_size,
                                   num_workers=self.num_workers, pin_memory=self.pin_memory)
    self.valid_loader = DataLoader(self.valid_dataset, batch_size=self.batch_size,
                                   num_workers=self.num_workers, pin_memory=self.pin_memory)
    self.test_loader = DataLoader(self.test_dataset, batch_size=self.batch_size,
                                   num_workers=self.num_workers, pin_memory=self.pin_memory)

def get_model(self):
    # Инициализация сиамской сети
    self.model = SiameseNetwork(self.name_model, self.emb_size, self.imagenet).

def get_opt_crit_sh(self):

```

```

self.criterion = ContrastiveLoss(margin=1.0).to(self.device)
# Определение оптимизатора
if self.start_learning_rate is not None:
    lr = self.start_learning_rate
else:
    lr = 0.001

self.optimizer = optim.__dict__[f"{self.name_optimizer}"](self.model.parameters())
# Создание планировщика LR
# ReduceLRonPlateau уменьшает скорость обучения, когда метрика перестает уменьшаться
self.scheduler = ReduceLRonPlateau(self.optimizer, mode='min', patience=2,

# Функция для обучения модели с валидацией
def train_model(self):

    train_loss_values = []
    valid_loss_values = []
    mAP_values = []

    for epoch in range(self.num_epochs):
        self.model.train()
        running_loss = 0.0
        for inputs1, inputs2, labels in tqdm(self.train_loader, desc=f"Epoch {epoch + 1}",
                                             unit="sample"):
            inputs1, inputs2, labels = inputs1.cuda(), inputs2.cuda(), labels.cuda()
            self.optimizer.zero_grad()
            emb1, emb2 = self.model(inputs1, inputs2)
            loss = self.criterion(emb1, emb2, labels)
            loss.backward()
            self.optimizer.step()
            running_loss += loss.item() * inputs1.size(0)

        # Вычисление Loss на валидационном датасете и метрик
        self.model.eval()
        valid_loss = 0.0
        best_mAP = 0.0
        all_embeddings = []
        all_labels = []
        with torch.no_grad():
            for inputs, labels in tqdm(self.valid_loader, desc=f"Epoch {epoch + 1}",
                                       unit="sample"):
                inputs, labels = inputs.cuda(), labels.cuda()
                outputs = self.model.forward_once(inputs)
                all_embeddings += [out.cpu().numpy() for out in outputs]
                all_labels += labels.cpu().numpy().tolist()

        epoch_loss = running_loss / len(self.train_loader.dataset)

        mAP = calculate_map(all_embeddings, all_labels) # Вычисление mAP для текущей эпохи

        # Сообщаем планировщику LR о текущей ошибке на валидационном наборе
        self.scheduler.step(mAP)

    # we want to save the model if the accuracy is the best
    if mAP > best_mAP:
        self.save_model()

```



```

        best_mAP = mAP

        # Добавление значений метрик в списки
        train_loss_values.append(epoch_loss)
        mAP_values.append(mAP)

        # Сохранение метрик
        self.save_metrics_train(train_loss_values,
                                mAP_values)

        print(f"\nEpoch {epoch + 1}/{self.num_epochs}, Training Loss: {epoch_loss}")
        print(f"mAP: {mAP}")

    print("Тренировка завершена!")

# Функция для оценки модели на тестовом датасете
def evaluate_model(self):
    self.model.eval()
    all_embeds = []
    all_labels = []

    with torch.no_grad():
        for inputs, labels in tqdm(self.valid_loader, desc=f"(Eval)",
                                    unit="sample"):
            inputs, labels = inputs.cuda(), labels.cuda()
            outputs = self.model.forward_once(inputs)
            all_embeds += [out.cpu().numpy() for out in outputs]
            all_labels += labels.cpu().numpy().tolist()

    mAP = calculate_map(all_embeds, all_labels) # Вычисление mAP для текущей эпохи
    print(f"Test mAP: {mAP}")

    self.save_metrics_test(mAP)

```

GraduateModelPipeline

In [340...

```

@dataclass
class GraduateModelPipeline:
    entry: EntryGraduateModel

    def __post_init__(self):
        pass

    def graduate(self):

        prefix = self.entry.Prefix
        models = self.entry.Models
        name_optimizers = self.entry.NameOptimizers
        is_use_imagenet_weights = self.entry.UseImageNetWeights
        path_to_data = self.entry.PathData
        path_to_weights = self.entry.PathWeights
        use_device = self.entry.UseDevice
        start_learning_rate = self.entry.StartLearningRate
        train_size_img = self.entry.SizeImg

```

```

emb_size = self.entry.EmbSize
batch_size = self.entry.BatchSize
num_workers = self.entry.NumWorkers
pin_memory = self.entry.PinMemory
num_epochs = self.entry.NumEpochs
seed = self.entry.Seed

path_to_metrics_train = self.entry.PathMetricsTrain
path_to_metrics_test = self.entry.PathMetricsTest
path_to_save_plots = self.entry.PathSavePlots

for index, name_model in enumerate(models):

    try:
        train = GraduateModel(
            prefix=prefix,
            name_model=name_model,
            path_to_data=path_to_data,
            path_to_weights=path_to_weights,
            path_to_metrics_train=path_to_metrics_train,
            path_to_metrics_test=path_to_metrics_test,
            is_use_imagenet_weights=is_use_imagenet_weights,
            name_optimizer=name_optimizers[index],
            num_epochs=num_epochs,
            batch_size=batch_size,
            train_size_img=train_size_img,
            emb_size=emb_size,
            use_device=use_device,
            num_workers=num_workers,
            pin_memory=pin_memory,
            seed=seed)

        train.graduate()
    except Exception as ex:
        log.exception("GraduateModel\n", exc_info=ex)

    # Построение графиков
    try:
        metrics_visualizer = MetricsVisualizer(prefix=prefix,
                                                path_to_metrics_train=path_to_me
                                                path_to_metrics_test=path_to_met
                                                path_to_save_plots=path_to_save_

    except Exception as ex:
        log.exception("MetricsVisualiser\n", exc_info=ex)

```

Визуализация графиков обучения

MetricsVisualizer

In [233...

```

class MetricsVisualizer:
    def __init__(self,
                 prefix: str = "",
                 path_to_metrics_train: str = "./metrics_train",

```

```

        path_to_metrics_test: str = "./metrics_test",
        path_to_save_plots: str = None):

    self.prefix = prefix
    if path_to_save_plots:
        os.makedirs(path_to_save_plots, exist_ok=True)
    self.path_to_save_plots = path_to_save_plots if path_to_save_plots is not None else None
    self.metrics_train_dir = os.path.join(path_to_metrics_train)
    self.metrics_test_dir = os.path.join(path_to_metrics_test)
    self.train_loss_values = {}
    self.score_values_valid = {}
    self.score_values_test = {}

    # Инициализируем сохранение графиков
    self.load_train_metrics()
    self.load_test_metrics()
    self.plot_metrics()

def _load_metrics(self, directory, files_dict, key_name):
    for file in os.listdir(directory):
        if file.endswith(".pt") and f"_{self.prefix}" in file:
            metrics = torch.load(os.path.join(directory, file))
            model_name = file.replace('.pt', '').replace(f'_{self.prefix}', '')
            files_dict[model_name] = metrics[key_name]

def load_train_metrics(self):
    self._load_metrics(self.metrics_train_dir, self.train_loss_values, 'train_loss')
    self._load_metrics(self.metrics_train_dir, self.score_values_valid, 'score')

def load_test_metrics(self):
    self._load_metrics(self.metrics_test_dir, self.score_values_test, 'score')

def plot_metrics(self):
    fig, axs = plt.subplots(3, 1, figsize=(14, 20))

    # Сравнение категориальной кроссэнтропии для разных моделей на тренировке
    for model, train_loss in self.train_loss_values.items():
        axs[0].plot(train_loss, label=model, linewidth=2)
    axs[0].set_title('Функция потерь на тренировке', fontsize=12)
    axs[0].set_xlabel('Эпоха')
    axs[0].set_ylabel('Значение функции потерь', fontsize=12)

    # Вычисление адаптивных лимитов для оси Y
    all_values = [value for values in self.train_loss_values.values() for value in values]
    lower_limit = np.percentile(all_values, 0) # нижний предел (например, 5-й перцентиль)
    upper_limit = np.percentile(all_values, 95) # верхний предел (например, 95-й перцентиль)

    try:
        # Применение ограничений
        axs[0].set_ylim(lower_limit, upper_limit)
    except Exception as ex:
        pass

    for model, score in self.score_values_valid.items():
        axs[1].plot(score, label=model, linewidth=2)
    axs[1].set_title('mAP на валидации', fontsize=12)

```


Epoch 1/10, Training Loss: 0.45255943820689026
mAP: 0.11211126853819527

```
Epoch 2/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.75sample/s]  
Epoch 2/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 32.85sample/s]
```

Epoch 2/10, Training Loss: 0.30936313100392
mAP: 0.11345065054626155

```
Epoch 3/10 (Train): 100%|███████████| 2  
56/256 [00:54<00:00, 4.70sample/s]  
Epoch 3/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 34.33sample/s]
```

Epoch 3/10, Training Loss: 0.2654756532574538
mAP: 0.10876011500995887

```
Epoch 4/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.76sample/s]  
Epoch 4/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 34.03sample/s]
```

Epoch 4/10, Training Loss: 0.2520573773654178
mAP: 0.11742499691248924

```
Epoch 5/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.78sample/s]  
Epoch 5/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 34.59sample/s]
```

Epoch 5/10, Training Loss: 0.24917301448294893
mAP: 0.1248071840936929

[illegible]

Epoch 00006: reducing learning rate of group 0 to 1.0000e-04.

Epoch 6/10, Training Loss: 0.24496416526380926
mAP: 0.12579843926683185

[illegible]

Epoch 7/10, Training Loss: 0.24577839352423325
mAP: 0.12780814555908623

```
Epoch 8/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.78sample/s]  
Epoch 8/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 34.43sample/s]
```

Epoch 8/10, Training Loss: 0.2460851634386927
mAP: 0.12438045531944177

```
Epoch 9/10 (Train): 100%|██████████| 2  
56/256 [00:54<00:00, 4.67sample/s]  
Epoch 9/10 (Eval): 100%|██████████|  
55/55 [00:01<00:00, 33.02sample/s]
```

Epoch 00009: reducing learning rate of group 0 to 1.0000e-05.

Epoch 9/10, Training Loss: 0.24497468868503347
mAP: 0.1299637832408077


```
Epoch 6/10 (Train): 100%|██████████████████████████████████████████████████████████████| 2  
56/256 [01:12<00:00, 3.51sample/s]  
Epoch 6/10 (Eval): 100%|██████████████████████████████████████████████████████████████|  
55/55 [00:02<00:00, 26.05sample/s]  
Epoch 6/10, Training Loss: 0.24521717394236475  
mAP: 0.13410158913243891
```

```
Epoch 7/10 (Train): 100%|██████████████████████████████████████████████████████████| 2  
56/256 [01:12<00:00, 3.54sample/s]  
Epoch 7/10 (Eval): 100%|██████████████████████████████████████████████████████████|  
55/55 [00:02<00:00, 25.47sample/s]  
Epoch 00007: reducing learning rate of group 0 to 1.0000e-05.
```

Epoch 7/10, Training Loss: 0.2469468319322914
mAP: 0.13567288851899695

```
Epoch 8/10 (Train): 100%|██████████████████████████████████████████████████████████| 2  
56/256 [01:12<00:00, 3.53sample/s]  
Epoch 8/10 (Eval): 100%|██████████████████████████████████████████████████████████|  
55/55 [00:02<00:00, 25.28sample/s]  
Epoch 8/10, Training Loss: 0.24437719851266593  
mAP: 0.12862555131842446
```

```
Epoch 9/10 (Train): 100%|██████████████████████████████████████████████████████████████| 2  
56/256 [01:12<00:00, 3.53sample/s]  
Epoch 9/10 (Eval): 100%|██████████████████████████████████████████████████████████████|  
55/55 [00:02<00:00, 26.52sample/s]  
Epoch 9/10, Training Loss: 0.24364256649278104  
mAP: 0.13300804481571513
```

```
Epoch 10/10 (Train): 100%|██████████████████████████████████████████████████████████| 2  
56/256 [01:11<00:00, 3.56sample/s]  
Epoch 10/10 (Eval): 100%|██████████████████████████████████████████████████████████|  
55/55 [00:02<00:00, 26.04sample/s]  
Epoch 00010: reducing learning rate of group 0 to 1.0000e-06.
```

Epoch 10/10, Training Loss: 0.24283002462470904
mAP: 0.1353440010786441
Тренировка завершена!

```
(Eval): 100% ██████████  
55/55 [00:02<00:00, 25.11sample/s]  
Test mAP: 0.1353440010786441
```

Выбранная модель: densenet201
 Пользовательское название модели: densenet201_Exp1
 Выбранный оптимизатор: SGD

[illegible]

Epoch 1/10, Training Loss: 0.316066330186359
mAP: 0.11660693069963368

```
Epoch 2/10 (Train): 100%|███████████| 2  
56/256 [01:25<00:00, 3.01sample/s]  
Epoch 2/10 (Eval): 100%|███████████|  
55/55 [00:02<00:00, 21.82sample/s]
```

Epoch 2/10, Training Loss: 0.27313150046393275
mAP: 0.11903601507097364

```
Epoch 3/10 (Train): 100%|███████████| 2  
56/256 [01:25<00:00, 3.00sample/s]  
Epoch 3/10 (Eval): 100%|███████████|  
55/55 [00:02<00:00, 22.74sample/s]
```

Epoch 3/10, Training Loss: 0.2555298166698776
mAP: 0.12120660896055012

```
Epoch 4/10 (Train): 100%|██████████| 2  
56/256 [01:24<00:00, 3.03sample/s]  
Epoch 4/10 (Eval): 100%|██████████|  
55/55 [00:02<00:00, 22.95sample/s]
```

Epoch 00004: reducing learning rate of group 0 to 1.0000e-04.

Epoch 4/10, Training Loss: 0.24595513253007084
mAP: 0.12333498081814272

[illegible]

Epoch 5/10, Training Loss: 0.24599000229500234
mAP: 0.12729650325152808

```
Epoch 6/10 (Train): 100%|███████████| 2  
56/256 [01:25<00:00, 2.99sample/s]  
Epoch 6/10 (Eval): 100%|███████████|  
55/55 [00:02<00:00, 22.66sample/s]
```

Epoch 6/10, Training Loss: 0.24656360049266368
mAP: 0.12487019021226968

```
Epoch 7/10 (Train): 100%|███████████| 2  
56/256 [01:25<00:00, 2.99sample/s]  
Epoch 7/10 (Eval): 100%|███████████|  
55/55 [00:02<00:00, 22.82sample/s]
```

Epoch 00007: reducing learning rate of group 0 to 1.0000e-05.

Epoch 7/10, Training Loss: 0.24745091702789068
mAP: 0.12789069393264518

```
Epoch 8/10 (Train): 100%|███████████| 2  
56/256 [01:25<00:00, 3.00sample/s]  
Epoch 8/10 (Eval): 100%|███████████|  
55/55 [00:02<00:00, 22.50sample/s]
```

Epoch 8/10, Training Loss: 0.24654953071149066
mAP: 0.13026280515054692

[illegible]

Epoch 9/10, Training Loss: 0.2446863838704303
mAP: 0.12747252676224138

Epoch 1/10, Training Loss: 0.42254169465013547
mAP: 0.08821747443343873

```
Epoch 2/10 (Train): 100%|██████████| 2  
56/256 [00:53<00:00, 4.83sample/s]  
Epoch 2/10 (Eval): 100%|██████████|  
55/55 [00:01<00:00, 43.99sample/s]
```

Epoch 2/10, Training Loss: 0.35572233450147905
mAP: 0.07076131857145755

```
Epoch 3/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.79sample/s]  
Epoch 3/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 44.62sample/s]
```

Epoch 3/10, Training Loss: 0.37660503200413586
mAP: 0.0796629590045794

```
Epoch 4/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.80sample/s]  
Epoch 4/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 44.77sample/s]
```

Epoch 4/10, Training Loss: 0.4137173338485809
mAP: 0.0789718454891387

[illegible]

Epoch 00005: reducing learning rate of group 0 to 1.0000e-04.

Epoch 5/10, Training Loss: 5.716999918244312
mAP: 0.07710725803359961

[illegible]

Epoch 6/10, Training Loss: 0.3725875879766818
mAP: 0.07341725404782788

```
Epoch 7/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.80sample/s]  
Epoch 7/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 39.38sample/s]
```

Epoch 7/10, Training Loss: 0.3364571274651098
mAP: 0.07137744342695201

```
Epoch 8/10 (Train): 100%|███████████| 2  
56/256 [00:54<00:00, 4.72sample/s]  
Epoch 8/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 42.28sample/s]
```

Epoch 8/10, Training Loss: 0.34347550584061537
mAP: 0.07045111578865859

```
Epoch 9/10 (Train): 100%|███████████| 2  
56/256 [00:53<00:00, 4.77sample/s]  
Epoch 9/10 (Eval): 100%|███████████|  
55/55 [00:01<00:00, 42.51sample/s]
```

Epoch 9/10, Training Loss: 0.34887090415577404
mAP: 0.07271640120041005


```

Traceback (most recent call last)
in <module>:1

> 1 graduate_pipeline.graduate()
  2

in graduate:48

    47                 seed=seed)
> 48         train.graduate()
    49         except Exception as ex:

in graduate:105

    104         # Обучаем
> 105         self.train_model()
    106         # Тестируем

in train_model:226

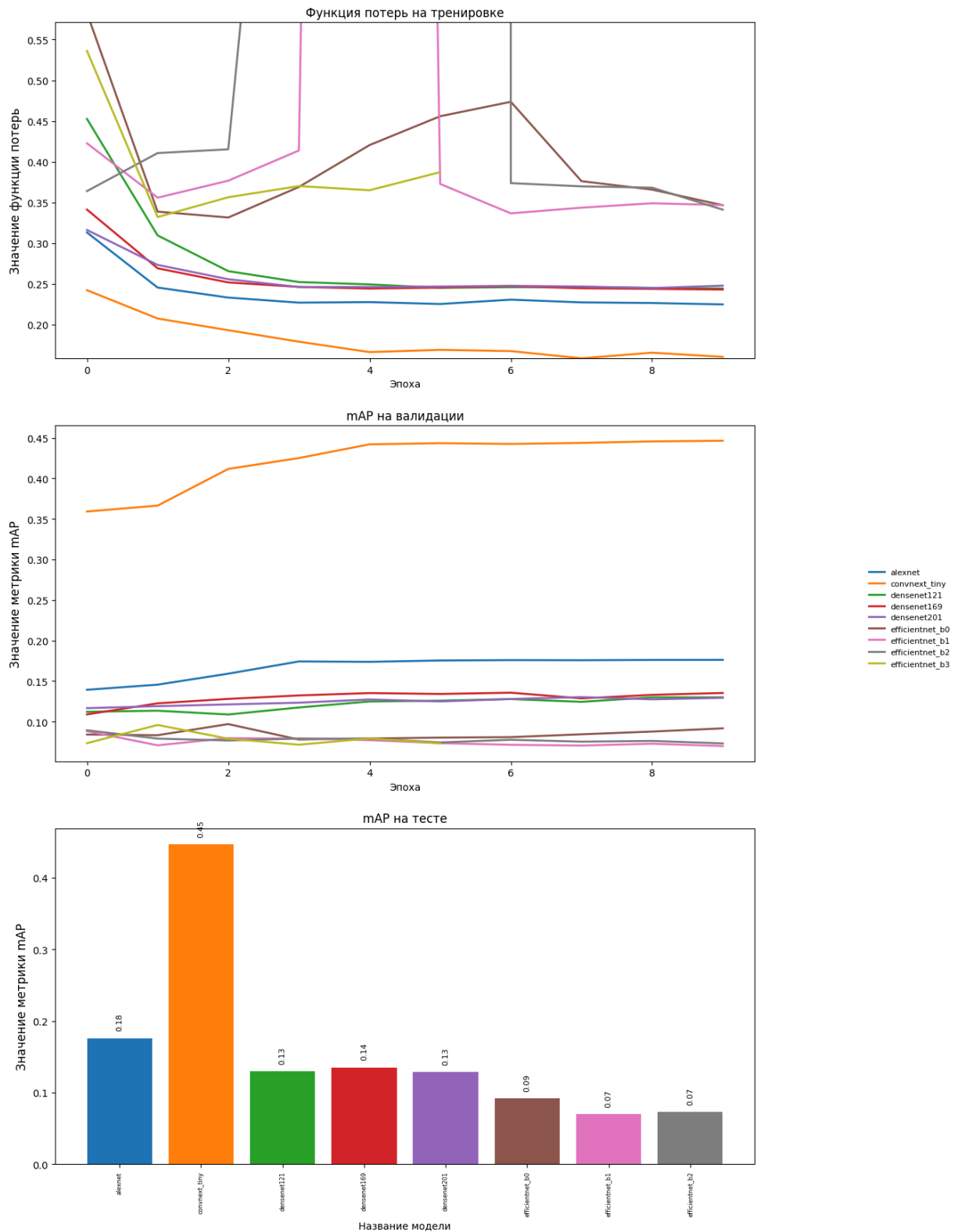
    225         inputs1, inputs2, labels = inputs1.cuda(), inputs2.cuda()
> 226         self.optimizer.zero_grad()
    227         emb1, emb2 = self.model(inputs1, inputs2)

c:\users\nightmare\pycharmprojects\cvtools\venv\lib\site-packages\torch\optim
    248         if (not foreach or p.grad.is_sparse):
> 249             p.grad.zero_()
    250         else:

```

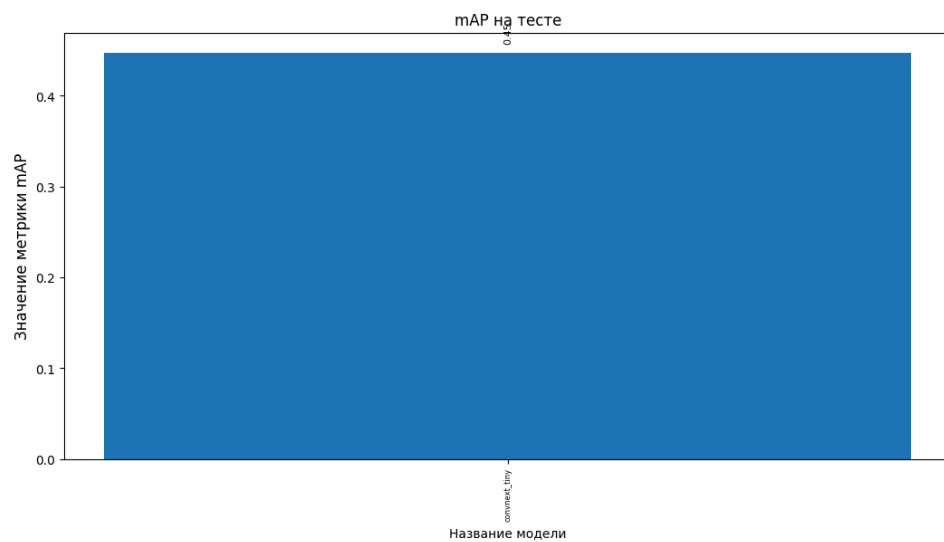
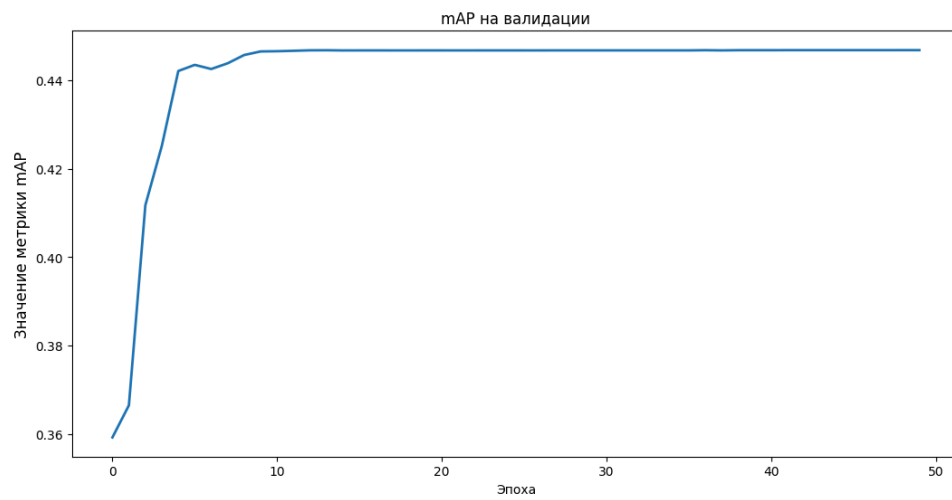
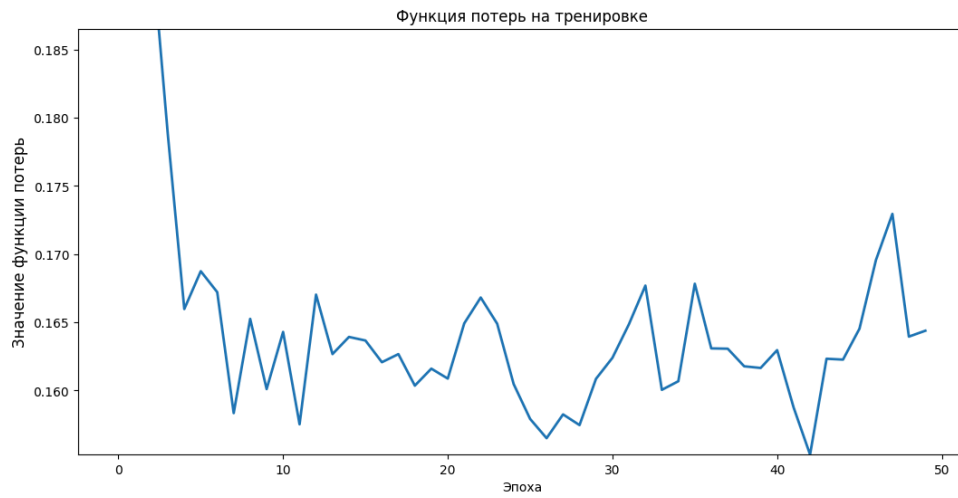
KeyboardInterrupt

[illegible]



Обучение лучшего бекбона

```
In [319... graduate_pipeline = validate_with_pydantic(EntryGraduateModel)(GraduateModelPipeline
entry = {
    "prefix": "Exp2",
    "models": ["convnext_tiny"],
    "name_optimizers": ["SGD"],
```

Тестирование модели

Инференс моделью

Класс InferenceModel

In [370...

```
class InferenceModel:
    def __init__(self,
                  model_name: str,
                  emb_size: int,
                  path_to_weights: str,
                  image_paths: list[str],
                  prefix: str = "",
                  use_device: str = None,
                  size_img: tuple = (128, 128),
                  labels: bool = False):
        self.name_model = model_name
        self.emb_size = emb_size
        self.path_to_weights = path_to_weights
        self.image_paths = image_paths
        self.size_img = size_img
        self.use_device = use_device
        self.device = None
        self.prefix = prefix
        self.labels = labels

        # Перемещение модели на GPU, если CUDA доступен
        if self.use_device == None:
            self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        elif self.use_device == "cuda":
            self.device = torch.device("cuda")
        elif self.use_device == "cpu":
            self.device = torch.device("cpu")
        else:
            raise Exception("use_device must be cuda or cpu or None")

    def get_model(self):
        # Инициализация сиамской сети
        self.model = SiameseNetwork(self.name_model, self.emb_size, False).to(self.device)

    def load_model(self):
        if self.prefix != "":
            path = os.path.join(self.path_to_weights, f"{self.name_model}_{self.prefix}")
        else:
            path = os.path.join(self.path_to_weights, f"{self.name_model}.pt")
        try:
            if self.name_model in models.__dict__:
                if os.path.exists(path):
                    state_dict = torch.load(path, map_location=self.device)['model_state_dict']
                    self.model.load_state_dict(state_dict)
                    log.info(f"Модель {self.name_model} успешно загружена с весами")
                else:
                    log.warning(f"Модель {self.name_model} не найдена.")
            except Exception as e:
                log.exception(f"Ошибка при загрузке модели: {e}")

    def get_transform(self):
        return transforms.Compose([
            ResizeWithPadding(self.size_img[0]),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.422], [0.229, 0.224, 0.225])])
```

```

        mean=[0.485, 0.456, 0.406], # Средние значения для ImageNet
        std=[0.229, 0.224, 0.225]   # Стандартные отклонения для ImageNet
    )
])

def predict(self):
    self.get_model()
    self.load_model()
    self.model.eval()
    all_embeds = []
    all_labels = []

    with torch.no_grad():
        for image_path in tqdm(self.image_paths, desc="Extract Feature", unit="s"):
            try:
                # Load image
                image = Image.open(image_path).convert('RGB')
                image = self.get_transform()(image).unsqueeze(0).to(self.device)

                # Get Label
                if self.labels:
                    label = int(os.path.basename(image_path).split(".png")[0].split("_")[0])
                else:
                    label = None

                all_labels.append(label)

                # Inference
                outputs = self.model.forward_once(image)
                all_embeds += [out.cpu().numpy() for out in outputs]

            except Exception as e:
                print(f"Ошибка при обработке изображения {image_path}: {e}")
                all_embeds.append(None)

    if self.labels:
        mAP = calculate_map(all_embeds, all_labels) # Вычисление mAP
        print(f"Inference mAP: {mAP}")

    return all_embeds, all_labels

```

InferenceModelPipeline

In [371...

```

@dataclass
class InferenceModelPipeline:
    entry: EntryInferenceModel

    def __post_init__(self):
        pass

    def inference(self):

        prefix = self.entry.Prefix
        name_model = self.entry.NameModel
        path_to_weights = self.entry.PathWeights

```


In [375...

```
class Emb2Tsne:
    def __init__(
        self,
        embeddings: list[np.ndarray],
        labels: List[int],
        n_components: int = 2,
        perplexity: float = 30.0,
        random_state: int = 42,
        **tsne_kwargs
    ):
        """
        Инициализация визуализатора t-SNE

        :param embeddings: Массив эмбедингов (N_samples, 512)
        :param labels: Список меток классов для каждого эмбединга
        :param n_components: Размерность выходного пространства
        :param perplexity: Параметр перплексии для t-SNE
        :param random_state: Seed для воспроизводимости
        :param tsne_kwargs: Дополнительные параметры для TSNE
        """
        self.embeddings = np.stack(embeddings)
        self.labels = np.array(labels)
        self.n_components = n_components
        self.tsne = TSNE(
            n_components=n_components,
            perplexity=perplexity,
            random_state=random_state,
            **tsne_kwargs
        )
        self.tsne_results: Optional[np.ndarray] = None

    def fit_transform(self) -> np.ndarray:
        """Применяет t-SNE к эмбедингам и возвращает 2D проекцию"""
        self.tsne_results = self.tsne.fit_transform(self.embeddings)
        return self.tsne_results

    def plot(
        self,
        figsize: tuple = (12, 8),
        title: str = "t-SNE Visualization",
        alpha: float = 0.7,
        marker_size: float = 8.0
    ) -> None:
        """Визуализирует 2D проекцию с цветовой маркировкой классов"""
        if self.tsne_results is None:
            raise ValueError("Сначала выполните fit_transform()")

        if len(np.unique(self.labels)) > 10:
            cmap = plt.cm.get_cmap('tab20')
        else:
            cmap = plt.cm.get_cmap('tab10')

        plt.figure(figsize=figsize)
        scatter = plt.scatter(
            self.tsne_results[:, 0],
```



```

        self.tsne_results[:, 1],
        c=self.labels,
        cmap=cmap,
        alpha=alpha,
        s=marker_size,
        edgecolors='w'
    )

    plt.title(title, pad=20)
    plt.xlabel("t-SNE Dimension 1")
    plt.ylabel("t-SNE Dimension 2")
    plt.colorbar(scatter, label="Classes")
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

```

Визуализируем

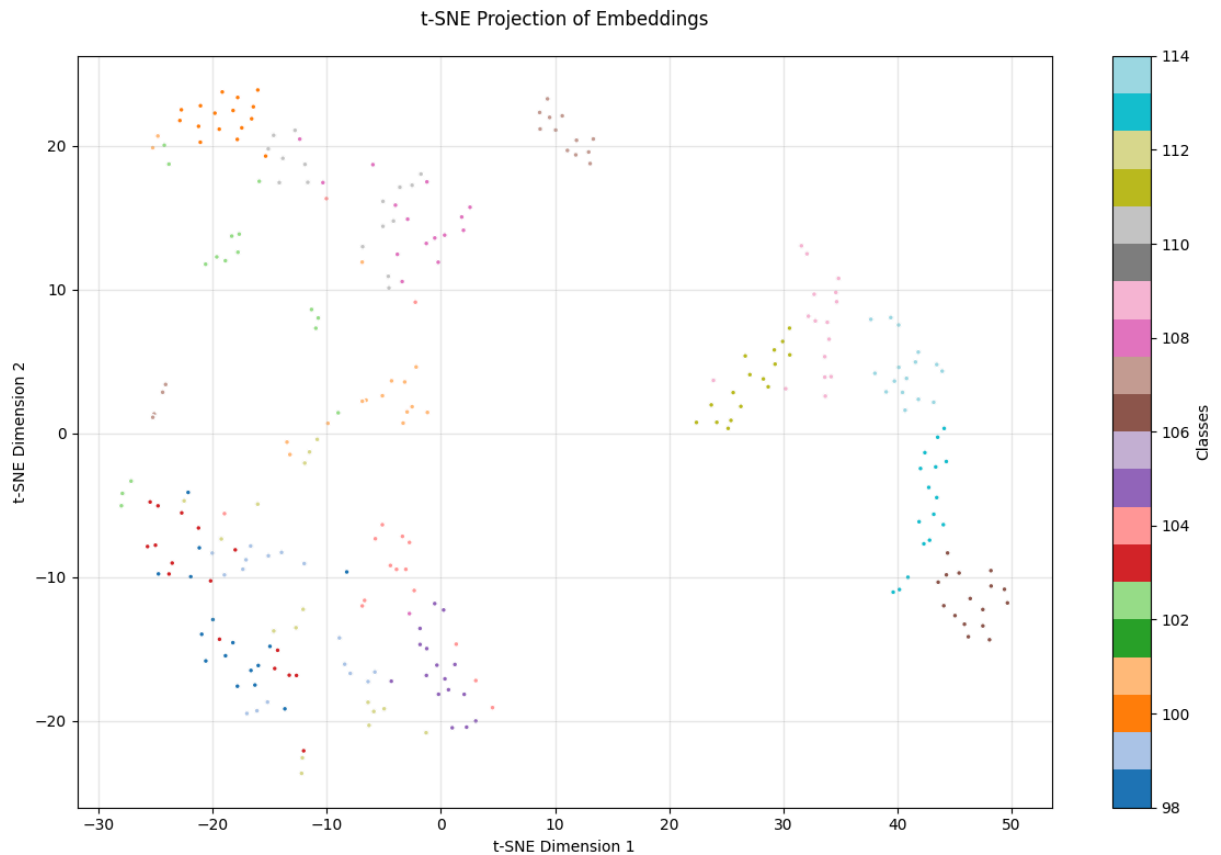
```

In [378... # Создание и использование визуализатора
visualizer = Emb2Tsne(
    embeddings=all_embeds,
    labels=all_labels,
    perplexity=15,
    random_state=42
)

# Получение 2D проекции
projection = visualizer.fit_transform()

# Визуализация
visualizer.plot(
    title="t-SNE Projection of Embeddings",
    alpha=1.0,
    marker_size=12.0
)

```



Реализация нейросетевого трекинга

Класс Tracker

```
In [390... TrackElement = collections.namedtuple(
    typename='TrackElement',
    field_names=['t', 'track_id', 'mask', 'mean', 'embed'],
)

def center_mass(masks: np.ndarray) -> np.ndarray:
    N = masks.shape[0]
    center_mass = np.empty((N, 2), dtype=np.float32)
    for i in range(N):
        xs, ys = masks[i].nonzero()
        center_mass[i, 0] = xs.mean()
        center_mass[i, 1] = ys.mean()
    return center_mass

class Tracker:
    def __init__(
        self,
        alive_threshold: int,
        association_threshold: float = 0.1,
        means_threshold: float = 150,
        min_value: float = -1e9,
        euclidean_scale: float = 1.344,
```

```

        euclidean_offset: float = 9.447,
        iou_scale: float = 0.402,
    ) -> None:
        self.alive_threshold = alive_threshold
        self.means_threshold = means_threshold
        self.association_threshold = association_threshold
        self.min_value = min_value
        self.active_tracks = []
        self.next_inst_id = None
        self.euclidean_scale = euclidean_scale
        self.euclidean_offset = euclidean_offset
        self.iou_scale = iou_scale

    def update_active_track(self, frame_count: int) -> None:
        self.active_tracks = [
            track for track in self.active_tracks
            if track.t >= frame_count - self.alive_threshold
        ]

    def tracking(
        self,
        frame_count: int,
        masks: np.ndarray,
        embeds: np.ndarray,
    ) -> list:
        if self.next_inst_id is None:
            self.next_inst_id = 1
        else:
            self.update_active_track(frame_count)

        n, k = len(masks), len(self.active_tracks)
        result = np.zeros(n, dtype=np.int32)

        if n < 1:
            return result

        means = center_mass(masks)
        masks = [maskUtils.encode(np.asfortranarray(mask)) for mask in masks.astype]

        if k == 0:
            for i in range(n):
                self.active_tracks.append(
                    TrackElement(
                        t=frame_count,
                        mask=masks[i],
                        track_id=self.next_inst_id,
                        mean=means[i],
                        embed=embeds[i],
                    )
                )
            result[i] = self.next_inst_id
            self.next_inst_id += 1
        return result

    detections_unassigned = np.ones(n, dtype=bool)

```

```

# Получаем эмбединги из памяти
last_reids = np.array([track.embed for track in self.active_tracks])

# Вычисляем матрицу схожести для эмбедингов
distance_matrix = cdist(embeds, last_reids, metric='euclidean')
embed_sim = self.euclidean_scale * (self.euclidean_offset - distance_matrix)

# Вычисляем матрицу IoU
memory_masks = [track.mask for track in self.active_tracks]
iou_matrix = maskUtils.iou(masks, memory_masks, [False]*len(memory_masks))
asso_sim = embed_sim + self.iou_scale * iou_matrix

# Применяем пороги и решаем задачу назначения
asso_sim[asso_sim <= self.association_threshold] = self.min_value
rows, cols = linear_sum_assignment(asso_sim, maximize=True)

for row, column in zip(rows, cols):
    if np.linalg.norm(self.active_tracks[column].mean - means[row]) < self.
        self.active_tracks[column] = TrackElement(
            t=frame_count,
            mask=masks[row],
            track_id=self.active_tracks[column].track_id,
            mean=means[row],
            embed=embeds[row],
        )
        result[row] = self.active_tracks[column].track_id
    else:
        self.active_tracks.append(TrackElement(
            t=frame_count,
            mask=masks[row],
            track_id=self.next_inst_id,
            mean=means[row],
            embed=embeds[row],
        ))
        result[row] = self.next_inst_id
        self.next_inst_id += 1
        detections_unassigned[row] = False

for i in detections_unassigned.nonzero()[0]:
    self.active_tracks.append(TrackElement(
        t=frame_count,
        mask=masks[i],
        track_id=self.next_inst_id,
        mean=means[i],
        embed=embeds[i],
    ))
    result[i] = self.next_inst_id
    self.next_inst_id += 1

return result

```

Функция для получения эмбедингов по известным детекциям

In [391...

```
def get_embeddings(
    img: np.ndarray,
    preds_bboxes: np.ndarray,
    emb_size: int,
    size_img: Tuple[int, int],
    prefix: str,
    name_model: str,
    path_to_weights: str = "./weights"
) -> np.ndarray:
    ...

    Функция для получения эмбедингов изображения по bounding boxes.

    Входные параметры:
    img - np.ndarray: Входное изображение (H, W, C).
    preds_bboxes - np.ndarray: Предсказанные bounding boxes (N, 4) в формате [x1, y
    emb_size - int: Размер эмбедингов модели.
    size_img - Tuple[int, int]: Размер изображений, на котором обучалась модель (H,
    prefix - str: Префикс модели.
    name_model - str: Название модели.
    path_to_weights - str: Путь к весам модели.

    Выходные параметры:
    np.ndarray: Эмбединги для каждого bounding box (N, emb_size).
    ...

    # Создаем временную директорию для сохранения обрезанных изображений
    with tempfile.TemporaryDirectory() as temp_dir:
        image_paths = []

        # Обрезаем изображение по каждому bounding box и сохраняем во временную дир
        for i, bbox in enumerate(preds_bboxes):
            x1, y1, x2, y2 = map(int, bbox)
            cropped_img = img[y1:y2, x1:x2] # Обрезаем изображение
            cropped_img = Image.fromarray(cropped_img) # Конвертируем в PIL Image
            cropped_img = cropped_img.resize(size_img) # Ресайзим до нужного разме

            # Сохраняем обрезанное изображение во временную директорию
            temp_image_path = os.path.join(temp_dir, f"cropped_{i}.jpg")
            cropped_img.save(temp_image_path)
            image_paths.append(temp_image_path)

        # Инициализируем модель для получения эмбедингов
        inference_model = InferenceModel(
            prefix=prefix,
            model_name=name_model,
            path_to_weights=path_to_weights,
            image_paths=image_paths,
            emb_size=emb_size,
            use_device="cuda",
            size_img=size_img,
            labels=False
        )

        # Получаем эмбединги
        all_embeds, all_labels = inference_model.predict()
```

```
# Возвращаем эмбединги в виде numpy массива
return np.stack(all_embeds, axis=0)
```

Скрипты для визуализации предсказаний

In [392...

```
# код с семинара
class YoloDetector:
    def __init__(self, conf_threshold: float = 0.3) -> None:
        self.model = YOLO("yolov8s-seg.pt")
        self.conf_threshold = conf_threshold

    def __call__(self, img_path: np.ndarray) -> np.ndarray:
        preds = self.model.predict(source=img_path, conf=self.conf_threshold, show=
        preds_masks = preds[0].masks.data.to(torch.bool).cpu().numpy()
        preds_bboxes = preds[0].boxes.xyxy.cpu().numpy()
        return preds_masks, preds_bboxes

def tracking_visualization(
    img: np.ndarray,
    preds: np.ndarray,
) -> np.ndarray:
    """
    Входные данные:
    img: np.ndarray - изображение для отрисовки
    preds: np.ndarray - маски сегментации
    """
    for i in range(len(preds)):
        # отрисовываем ограничивающий прямоугольник
        cv2.rectangle(
            img,
            (int(preds[i][0]), int(preds[i][1])),
            (int(preds[i][2]), int(preds[i][3])),
            color=(0, 255, 0),
            thickness=1,
        )
        # отрисовываем track_id
        cv2.putText(
            img,
            str(int(preds[i][4])),
            (int(preds[i][0] + 10), int(preds[i][1] + 10)),
            cv2.FONT_HERSHEY_SIMPLEX,
            fontScale=1,
            color=(0, 255, 0),
            thickness=1,
        )

    return img
```

Оцениваем работу трекера

Скачиваем видео

In [384...

```
# скачаем тестовое видео
```

```
!wget https://motchallenge.net/sequenceVideos/MOT20-02-raw.mp4 -O mot20.mp4
```

```
--2025-02-25 13:17:11-- https://motchallenge.net/sequenceVideos/MOT20-02-raw.mp4
Resolving motchallenge.net (motchallenge.net)... 131.159.19.34, 2a09:80c0:18::1034
Connecting to motchallenge.net (motchallenge.net)|131.159.19.34|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 76066403 (73M) [video/mp4]
Saving to: 'mot20.mp4'
```

0K		0%	453K	2m44s
50K		0%	960K	2m1s
100K		0%	33,6M	81s
150K		0%	20,5M	62s
200K		0%	1023K	64s
250K		0%	926K	66s
300K		0%	91,5M	57s
350K		0%	678M	50s
400K		0%	700M	44s
450K		0%	937K	48s
500K		0%	19,9M	44s
550K		0%	21,6M	40s
600K		0%	739M	37s
650K		0%	1,01M	40s
700K		1%	929K	42s
750K		1%	923K	45s
800K		1%	227M	42s
850K		1%	886M	40s
900K		1%	9,04M	38s
950K		1%	1,03M	39s
1000K		1%	4,80M	38s
1050K		1%	6,18M	37s
1100K		1%	1,35M	38s
1150K		1%	5,49M	37s
1200K		1%	3,95M	36s
1250K		1%	1,38M	36s
1300K		1%	9,39M	35s
1350K		1%	3,95M	35s
1400K		1%	1,47M	35s
1450K		2%	6,61M	34s
1500K		2%	3,97M	34s
1550K		2%	1,47M	34s
1600K		2%	4,34M	34s
1650K		2%	5,38M	33s
1700K		2%	1,38M	33s
1750K		2%	10,6M	33s
1800K		2%	3,91M	32s
1850K		2%	1,48M	33s
1900K		2%	6,82M	32s
1950K		2%	3,84M	32s
2000K		2%	1,36M	32s
2050K		2%	9,51M	32s
2100K		2%	4,08M	31s
2150K		2%	1,50M	32s
2200K		3%	6,14M	31s
2250K		3%	5,32M	31s
2300K		3%	1,45M	31s
2350K		3%	5,71M	31s
2400K		3%	3,52M	30s

2450K						3%	1,58M	31s
2500K						3%	4,44M	30s
2550K						3%	8,68M	30s
2600K						3%	1,33M	30s
2650K						3%	6,97M	30s
2700K						3%	5,30M	30s
2750K						3%	5,79M	29s
2800K						3%	1,42M	30s
2850K						3%	4,90M	29s
2900K						3%	5,79M	29s
2950K						4%	1,41M	29s
3000K						4%	7,69M	29s
3050K						4%	5,36M	29s
3100K						4%	1,46M	29s
3150K						4%	5,62M	29s
3200K						4%	4,20M	28s
3250K						4%	1,48M	29s
3300K						4%	6,01M	28s
3350K						4%	6,45M	28s
3400K						4%	1,33M	28s
3450K						4%	10,5M	28s
3500K						4%	5,46M	28s
3550K						4%	5,87M	28s
3600K						4%	1,37M	28s
3650K						4%	5,43M	28s
3700K						5%	5,74M	28s
3750K						5%	1,37M	28s
3800K						5%	6,58M	28s
3850K						5%	2,90M	27s
3900K						5%	1,87M	28s
3950K						5%	5,86M	27s
4000K						5%	3,35M	27s
4050K						5%	1,63M	27s
4100K						5%	15,0M	27s
4150K						5%	3,04M	27s
4200K						5%	4,24M	27s
4250K						5%	1,97M	27s
4300K						5%	5,27M	27s
4350K						5%	3,29M	27s
4400K						5%	1,70M	27s
4450K						6%	8,09M	27s
4500K						6%	2,69M	27s
4550K						6%	1,85M	27s
4600K						6%	5,34M	27s
4650K						6%	4,85M	26s
4700K						6%	3,07M	26s
4750K						6%	2,10M	26s
4800K						6%	4,33M	26s
4850K						6%	3,41M	26s
4900K						6%	1,80M	26s
4950K						6%	9,09M	26s
5000K						6%	2,91M	26s
5050K						6%	2,03M	26s
5100K						6%	7,07M	26s
5150K						7%	4,36M	26s
5200K						7%	2,72M	26s

5250K						7%	2,00M	26s
5300K						7%	5,76M	26s
5350K						7%	2,96M	26s
5400K						7%	1,78M	26s
5450K						7%	13,8M	25s
5500K						7%	2,80M	25s
5550K						7%	6,87M	25s
5600K						7%	1,72M	25s
5650K						7%	4,21M	25s
5700K						7%	3,84M	25s
5750K						7%	1,92M	25s
5800K						7%	12,9M	25s
5850K						7%	2,94M	25s
5900K						8%	1,61M	25s
5950K						8%	8,76M	25s
6000K						8%	2,84M	25s
6050K						8%	5,85M	25s
6100K						8%	2,02M	25s
6150K						8%	6,11M	25s
6200K						8%	2,84M	25s
6250K						8%	1,78M	25s
6300K						8%	8,13M	25s
6350K						8%	5,08M	25s
6400K						8%	1,39M	25s
6450K						8%	18,7M	25s
6500K						8%	4,63M	24s
6550K						8%	2,71M	24s
6600K						8%	2,04M	24s
6650K						9%	7,88M	24s
6700K						9%	2,82M	24s
6750K						9%	1,86M	24s
6800K						9%	5,33M	24s
6850K						9%	5,18M	24s
6900K						9%	3,71M	24s
6950K						9%	1,95M	24s
7000K						9%	8,30M	24s
7050K						9%	2,80M	24s
7100K						9%	1,86M	24s
7150K						9%	5,16M	24s
7200K						9%	2,93M	24s
7250K						9%	1,82M	24s
7300K						9%	15,2M	24s
7350K						9%	7,21M	24s
7400K						10%	2,88M	24s
7450K						10%	1,71M	24s
7500K						10%	7,07M	24s
7550K						10%	5,76M	24s
7600K						10%	1,39M	24s
7650K						10%	13,7M	24s
7700K						10%	5,72M	23s
7750K						10%	3,26M	23s
7800K						10%	1,73M	23s
7850K						10%	7,15M	23s
7900K						10%	2,71M	23s
7950K						10%	7,19M	23s
8000K						10%	2,22M	23s

8050K						10%	6,01M	23s
8100K						10%	2,50M	23s
8150K						11%	2,02M	23s
8200K						11%	6,72M	23s
8250K						11%	2,66M	23s
8300K						11%	638K	24s
8350K						11%	199M	23s
8400K						11%	5,30M	23s
8450K						11%	538M	23s
8500K						11%	757M	23s
8550K						11%	2,65M	23s
8600K						11%	1,79M	23s
8650K						11%	6,07M	23s
8700K						11%	1,15M	23s
8750K						11%	5,15M	23s
8800K						11%	2,38M	23s
8850K						11%	1,94M	23s
8900K						12%	3,20M	23s
8950K						12%	1,37M	23s
9000K						12%	5,12M	23s
9050K						12%	2,70M	23s
9100K						12%	1,94M	23s
9150K						12%	4,65M	23s
9200K						12%	2,63M	23s
9250K						12%	1,36M	23s
9300K						12%	9,86M	23s
9350K						12%	1,37M	23s
9400K						12%	3,98M	23s
9450K						12%	2,95M	23s
9500K						12%	1,99M	23s
9550K						12%	3,87M	23s
9600K						12%	2,72M	23s
9650K						13%	1,68M	23s
9700K						13%	4,73M	23s
9750K						13%	1,34M	23s
9800K						13%	4,03M	23s
9850K						13%	10,2M	23s
9900K						13%	1,35M	23s
9950K						13%	4,10M	23s
10000K						13%	2,49M	23s
10050K						13%	2,23M	23s
10100K						13%	4,26M	23s
10150K						13%	2,48M	23s
10200K						13%	2,22M	23s
10250K						13%	3,06M	23s
10300K						13%	3,25M	23s
10350K						14%	2,26M	23s
10400K						14%	2,77M	23s
10450K						14%	3,40M	23s
10500K						14%	1,98M	23s
10550K						14%	3,55M	23s
10600K						14%	3,33M	23s
10650K						14%	2,29M	23s
10700K						14%	2,88M	23s
10750K						14%	3,35M	23s
10800K						14%	1,89M	23s

10850K						14%	4,12M	23s
10900K						14%	3,16M	23s
10950K						14%	673K	23s
11000K						14%	700M	23s
11050K						14%	547M	23s
11100K						15%	18,0M	23s
11150K						15%	1,72M	23s
11200K						15%	1,49M	23s
11250K						15%	2,39M	23s
11300K						15%	2,00M	23s
11350K						15%	1,69M	23s
11400K						15%	2,26M	23s
11450K						15%	1,61M	23s
11500K						15%	2,17M	23s
11550K						15%	3,17M	23s
11600K						15%	1,30M	23s
11650K						15%	3,20M	23s
11700K						15%	2,73M	23s
11750K						15%	1,58M	23s
11800K						15%	2,26M	23s
11850K						16%	2,14M	23s
11900K						16%	1,64M	23s
11950K						16%	673K	23s
12000K						16%	397M	23s
12050K						16%	770M	23s
12100K						16%	945K	23s
12150K						16%	4,80M	23s
12200K						16%	1,14M	23s
12250K						16%	1,56M	23s
12300K						16%	2,20M	23s
12350K						16%	954K	23s
12400K						16%	4,98M	23s
12450K						16%	1,12M	23s
12500K						16%	1,60M	23s
12550K						16%	2,17M	23s
12600K						17%	952K	24s
12650K						17%	5,07M	24s
12700K						17%	1,12M	24s
12750K						17%	5,31M	24s
12800K						17%	1,10M	24s
12850K						17%	816K	24s
12900K						17%	284M	24s
12950K						17%	949K	24s
13000K						17%	1,10M	24s
13050K						17%	935K	24s
13100K						17%	1,60M	24s
13150K						17%	1,56M	24s
13200K						17%	934K	24s
13250K						17%	1,09M	24s
13300K						17%	941K	25s
13350K						18%	4,44M	24s
13400K						18%	982K	25s
13450K						18%	1,10M	25s
13500K						18%	1,58M	25s
13550K						18%	1,54M	25s
13600K						18%	937K	25s

13650K						18%	1,10M	25s
13700K						18%	940K	25s
13750K						18%	5,33M	25s
13800K						18%	944K	25s
13850K						18%	1,10M	25s
13900K						18%	1,49M	25s
13950K						18%	1,68M	25s
14000K						18%	873K	25s
14050K						18%	1,23M	25s
14100K						19%	3,52M	25s
14150K						19%	941K	25s
14200K						19%	1,25M	26s
14250K						19%	3,49M	25s
14300K						19%	1,24M	26s
14350K						19%	1,46M	26s
14400K						19%	964K	26s
14450K						19%	1,42M	26s
14500K						19%	1,24M	26s
14550K						19%	3,39M	26s
14600K						19%	1,26M	26s
14650K						19%	1,52M	26s
14700K						19%	1,40M	26s
14750K						19%	1,26M	26s
14800K						19%	1,52M	26s
14850K						20%	1,34M	26s
14900K						20%	1,31M	26s
14950K						20%	3,00M	26s
15000K						20%	991K	26s
15050K						20%	2,66M	26s
15100K						20%	1,33M	26s
15150K						20%	1,31M	26s
15200K						20%	1,46M	26s
15250K						20%	1,40M	26s
15300K						20%	2,32M	26s
15350K						20%	1,53M	26s
15400K						20%	1,29M	26s
15450K						20%	2,61M	26s
15500K						20%	1,42M	26s
15550K						21%	1,48M	26s
15600K						21%	1,29M	26s
15650K						21%	1,41M	26s
15700K						21%	2,32M	26s
15750K						21%	1,51M	26s
15800K						21%	2,37M	26s
15850K						21%	1,50M	26s
15900K						21%	1,50M	26s
15950K						21%	1,47M	26s
16000K						21%	1,22M	26s
16050K						21%	2,84M	26s
16100K						21%	1,36M	26s
16150K						21%	2,46M	26s
16200K						21%	1,47M	26s
16250K						21%	2,51M	26s
16300K						22%	1,45M	26s
16350K						22%	2,54M	26s
16400K						22%	1,07M	26s

16450K						22%	2,41M	26s
16500K						22%	1,28M	26s
16550K						22%	3,31M	26s
16600K						22%	1,29M	26s
16650K						22%	3,27M	26s
16700K						22%	1,28M	26s
16750K						22%	3,28M	26s
16800K						22%	1,27M	26s
16850K						22%	1,36M	26s
16900K						22%	2,87M	26s
16950K						22%	1,36M	26s
17000K						22%	2,86M	26s
17050K						23%	1,36M	26s
17100K						23%	2,86M	26s
17150K						23%	1,35M	26s
17200K						23%	2,84M	26s
17250K						23%	1,36M	26s
17300K						23%	2,77M	26s
17350K						23%	1,38M	26s
17400K						23%	2,18M	26s
17450K						23%	1,60M	26s
17500K						23%	2,52M	26s
17550K						23%	3,48M	26s
17600K						23%	1,17M	26s
17650K						23%	1,55M	26s
17700K						23%	2,26M	26s
17750K						23%	1,52M	26s
17800K						24%	2,66M	26s
17850K						24%	3,93M	26s
17900K						24%	1,50M	26s
17950K						24%	2,39M	26s
18000K						24%	1,19M	26s
18050K						24%	3,92M	26s
18100K						24%	1,50M	26s
18150K						24%	2,38M	26s
18200K						24%	1,51M	26s
18250K						24%	4,18M	26s
18300K						24%	1,58M	26s
18350K						24%	2,24M	26s
18400K						24%	1,57M	26s
18450K						24%	2,21M	26s
18500K						24%	1,59M	26s
18550K						25%	2,22M	26s
18600K						25%	4,58M	26s
18650K						25%	1,61M	26s
18700K						25%	2,12M	26s
18750K						25%	1,63M	26s
18800K						25%	2,21M	26s
18850K						25%	1,58M	26s
18900K						25%	3,93M	26s
18950K						25%	1,56M	26s
19000K						25%	2,24M	25s
19050K						25%	4,06M	25s
19100K						25%	1,70M	25s
19150K						25%	2,05M	25s
19200K						25%	1,67M	25s

19250K						25%	2,05M	25s
19300K						26%	2,35M	25s
19350K						26%	2,09M	25s
19400K						26%	1,66M	25s
19450K						26%	2,08M	25s
19500K						26%	5,52M	25s
19550K						26%	1,63M	25s
19600K						26%	2,14M	25s
19650K						26%	1,63M	25s
19700K						26%	3,02M	25s
19750K						26%	1,77M	25s
19800K						26%	2,05M	25s
19850K						26%	5,38M	25s
19900K						26%	1,65M	25s
19950K						26%	2,12M	25s
20000K						26%	1,63M	25s
20050K						27%	3,46M	25s
20100K						27%	1,63M	25s
20150K						27%	2,57M	25s
20200K						27%	3,07M	25s
20250K						27%	1,75M	25s
20300K						27%	3,53M	25s
20350K						27%	1,65M	25s
20400K						27%	2,09M	25s
20450K						27%	1,64M	25s
20500K						27%	5,32M	25s
20550K						27%	2,17M	25s
20600K						27%	2,30M	25s
20650K						27%	2,11M	25s
20700K						27%	4,33M	25s
20750K						28%	1,77M	25s
20800K						28%	2,14M	25s
20850K						28%	1,62M	25s
20900K						28%	3,43M	25s
20950K						28%	1,64M	25s
21000K						28%	5,40M	24s
21050K						28%	2,17M	24s
21100K						28%	2,21M	24s
21150K						28%	2,10M	24s
21200K						28%	1,64M	24s
21250K						28%	5,37M	24s
21300K						28%	2,23M	24s
21350K						28%	2,12M	24s
21400K						28%	2,13M	24s
21450K						28%	4,13M	24s
21500K						29%	1,83M	24s
21550K						29%	3,31M	24s
21600K						29%	1,60M	24s
21650K						29%	2,13M	24s
21700K						29%	4,24M	24s
21750K						29%	1,80M	24s
21800K						29%	3,36M	24s
21850K						29%	1,62M	24s
21900K						29%	4,04M	24s
21950K						29%	2,71M	24s
22000K						29%	1,57M	24s

22050K						29%	3,39M	24s
22100K						29%	4,46M	24s
22150K						29%	1,56M	24s
22200K						29%	2,71M	24s
22250K						30%	2,12M	24s
22300K						30%	2,79M	24s
22350K						30%	3,93M	24s
22400K						30%	1,54M	24s
22450K						30%	3,32M	24s
22500K						30%	1,64M	24s
22550K						30%	3,75M	24s
22600K						30%	4,45M	24s
22650K						30%	1,66M	24s
22700K						30%	3,85M	24s
22750K						30%	2,90M	23s
22800K						30%	1,53M	23s
22850K						30%	3,35M	23s
22900K						30%	4,86M	23s
22950K						30%	1,52M	23s
23000K						31%	4,46M	23s
23050K						31%	1,62M	23s
23100K						31%	4,19M	23s
23150K						31%	4,22M	23s
23200K						31%	1,61M	23s
23250K						31%	2,23M	23s
23300K						31%	7,51M	23s
23350K						31%	1,99M	23s
23400K						31%	2,25M	23s
23450K						31%	7,00M	23s
23500K						31%	1,56M	23s
23550K						31%	3,36M	23s
23600K						31%	1,59M	23s
23650K						31%	4,31M	23s
23700K						31%	2,82M	23s
23750K						32%	2,00M	23s
23800K						32%	4,39M	23s
23850K						32%	2,79M	23s
23900K						32%	1,99M	23s
23950K						32%	4,26M	23s
24000K						32%	2,79M	23s
24050K						32%	1,55M	23s
24100K						32%	3,53M	23s
24150K						32%	6,25M	23s
24200K						32%	1,56M	23s
24250K						32%	3,58M	23s
24300K						32%	5,02M	23s
24350K						32%	1,65M	22s
24400K						32%	3,34M	22s
24450K						32%	1,57M	22s
24500K						33%	4,14M	22s
24550K						33%	5,13M	22s
24600K						33%	1,55M	22s
24650K						33%	4,12M	22s
24700K						33%	5,11M	22s
24750K						33%	1,54M	22s
24800K						33%	3,09M	22s

24850K						33%	3,70M	22s
24900K						33%	2,01M	22s
24950K						33%	4,43M	22s
25000K						33%	2,73M	22s
25050K						33%	598K	22s
25100K						33%	3,36M	22s
25150K						33%	531M	22s
25200K						33%	4,51M	22s
25250K						34%	1,73M	22s
25300K						34%	2,15M	22s
25350K						34%	1,63M	22s
25400K						34%	2,26M	22s
25450K						34%	1,56M	22s
25500K						34%	5,24M	22s
25550K						34%	1,12M	22s
25600K						34%	3,39M	22s
25650K						34%	1,27M	22s
25700K						34%	5,42M	22s
25750K						34%	1,32M	22s
25800K						34%	6,41M	22s
25850K						34%	2,07M	22s
25900K						34%	1,66M	22s
25950K						35%	2,34M	22s
26000K						35%	1,55M	22s
26050K						35%	3,51M	22s
26100K						35%	1,24M	22s
26150K						35%	5,80M	22s
26200K						35%	1,29M	22s
26250K						35%	6,69M	22s
26300K						35%	2,34M	22s
26350K						35%	1,55M	22s
26400K						35%	2,30M	22s
26450K						35%	1,54M	22s
26500K						35%	19,0M	22s
26550K						35%	1,12M	22s
26600K						35%	6,71M	21s
26650K						35%	2,36M	21s
26700K						36%	1,54M	21s
26750K						36%	17,7M	21s
26800K						36%	1,11M	21s
26850K						36%	7,37M	21s
26900K						36%	2,19M	21s
26950K						36%	1,62M	21s
27000K						36%	5,72M	21s
27050K						36%	1,27M	21s
27100K						36%	7,55M	21s
27150K						36%	2,32M	21s
27200K						36%	1,53M	21s
27250K						36%	6,09M	21s
27300K						36%	1,26M	21s
27350K						36%	7,39M	21s
27400K						36%	2,31M	21s
27450K						37%	1,87M	21s
27500K						37%	7,12M	21s
27550K						37%	1,07M	21s
27600K						37%	7,46M	21s

27650K						37%	2,34M	21s
27700K						37%	1,85M	21s
27750K						37%	6,95M	21s
27800K						37%	2,50M	21s
27850K						37%	1,52M	21s
27900K						37%	6,42M	21s
27950K						37%	1,24M	21s
28000K						37%	7,28M	21s
28050K						37%	2,42M	21s
28100K						37%	1,81M	21s
28150K						37%	7,88M	21s
28200K						38%	1,05M	21s
28250K						38%	9,49M	21s
28300K						38%	6,56M	21s
28350K						38%	1,21M	21s
28400K						38%	8,93M	21s
28450K						38%	2,39M	21s
28500K						38%	1,69M	21s
28550K						38%	8,35M	20s
28600K						38%	2,68M	20s
28650K						38%	1,52M	20s
28700K						38%	6,53M	20s
28750K						38%	1,16M	20s
28800K						38%	8,95M	20s
28850K						38%	2,63M	20s
28900K						38%	1,66M	20s
28950K						39%	10,5M	20s
29000K						39%	2,57M	20s
29050K						39%	1,67M	20s
29100K						39%	9,64M	20s
29150K						39%	2,57M	20s
29200K						39%	1,50M	20s
29250K						39%	7,51M	20s
29300K						39%	1,15M	20s
29350K						39%	12,6M	20s
29400K						39%	4,77M	20s
29450K						39%	1,26M	20s
29500K						39%	11,6M	20s
29550K						39%	2,47M	20s
29600K						39%	1,65M	20s
29650K						39%	13,0M	20s
29700K						40%	2,43M	20s
29750K						40%	1,51M	20s
29800K						40%	48,3M	20s
29850K						40%	2,29M	20s
29900K						40%	1,60M	20s
29950K						40%	47,3M	20s
30000K						40%	1,01M	20s
30050K						40%	10,5M	20s
30100K						40%	5,27M	20s
30150K						40%	1,11M	20s
30200K						40%	23,6M	20s
30250K						40%	835K	20s
30300K						40%	718M	20s
30350K						40%	800M	19s
30400K						40%	1,09M	20s

30450K						41%	2,40M	19s
30500K						41%	1,54M	19s
30550K						41%	9,60M	19s
30600K						41%	1,02M	19s
30650K						41%	852K	19s
30700K						41%	451M	19s
30750K						41%	948K	19s
30800K						41%	420M	19s
30850K						41%	943K	19s
30900K						41%	1,02M	19s
30950K						41%	8,59M	19s
31000K						41%	956K	19s
31050K						41%	1,03M	19s
31100K						41%	8,27M	19s
31150K						42%	1,03M	19s
31200K						42%	952K	19s
31250K						42%	8,44M	19s
31300K						42%	952K	19s
31350K						42%	1,07M	19s
31400K						42%	6,51M	19s
31450K						42%	1,04M	19s
31500K						42%	6,85M	19s
31550K						42%	966K	19s
31600K						42%	1,04M	19s
31650K						42%	6,98M	19s
31700K						42%	955K	19s
31750K						42%	1,06M	19s
31800K						42%	6,84M	19s
31850K						42%	501K	19s
31900K						43%	204M	19s
31950K						43%	7,06M	19s
32000K						43%	503K	19s
32050K						43%	7,40M	19s
32100K						43%	927K	19s
32150K						43%	942K	19s
32200K						43%	1,03M	19s
32250K						43%	934K	19s
32300K						43%	8,02M	19s
32350K						43%	944K	19s
32400K						43%	926K	19s
32450K						43%	942K	19s
32500K						43%	1,06M	19s
32550K						43%	5,78M	19s
32600K						43%	961K	19s
32650K						44%	942K	19s
32700K						44%	1,07M	19s
32750K						44%	6,29M	19s
32800K						44%	943K	19s
32850K						44%	953K	19s
32900K						44%	1,07M	19s
32950K						44%	945K	19s
33000K						44%	6,12M	19s
33050K						44%	942K	19s
33100K						44%	1,08M	19s
33150K						44%	5,84M	19s
33200K						44%	954K	19s

33250K						44%	956K	19s
33300K						44%	1,06M	19s
33350K						44%	5,55M	19s
33400K						45%	955K	19s
33450K						45%	1,08M	19s
33500K						45%	5,95M	19s
33550K						45%	954K	19s
33600K						45%	1,07M	19s
33650K						45%	6,11M	19s
33700K						45%	955K	19s
33750K						45%	1,07M	19s
33800K						45%	6,06M	19s
33850K						45%	502K	19s
33900K						45%	8,62M	19s
33950K						45%	940K	19s
34000K						45%	944K	19s
34050K						45%	944K	19s
34100K						45%	1,04M	19s
34150K						46%	944K	19s
34200K						46%	7,46M	19s
34250K						46%	962K	19s
34300K						46%	943K	19s
34350K						46%	1,03M	19s
34400K						46%	854K	19s
34450K						46%	1,03M	19s
34500K						46%	7,78M	19s
34550K						46%	953K	19s
34600K						46%	944K	19s
34650K						46%	1,03M	19s
34700K						46%	8,32M	19s
34750K						46%	964K	19s
34800K						46%	936K	19s
34850K						46%	1,01M	19s
34900K						47%	943K	19s
34950K						47%	9,74M	19s
35000K						47%	954K	19s
35050K						47%	1,00M	19s
35100K						47%	9,07M	19s
35150K						47%	947K	19s
35200K						47%	939K	19s
35250K						47%	1,03M	19s
35300K						47%	7,78M	19s
35350K						47%	954K	19s
35400K						47%	1,04M	19s
35450K						47%	7,29M	19s
35500K						47%	953K	19s
35550K						47%	1,04M	19s
35600K						47%	939K	19s
35650K						48%	6,92M	19s
35700K						48%	953K	19s
35750K						48%	1,05M	19s
35800K						48%	6,79M	19s
35850K						48%	474K	19s
35900K						48%	364M	19s
35950K						48%	948K	19s
36000K						48%	930K	19s

36050K						48%	958K	19s
36100K						48%	941K	19s
36150K						48%	942K	19s
36200K						48%	1,07M	19s
36250K						48%	6,14M	19s
36300K						48%	944K	19s
36350K						49%	943K	19s
36400K						49%	922K	19s
36450K						49%	944K	19s
36500K						49%	849K	19s
36550K						49%	989K	19s
36600K						49%	1,16M	19s
36650K						49%	3,08M	19s
36700K						49%	990K	19s
36750K						49%	952K	19s
36800K						49%	947K	19s
36850K						49%	988K	19s
36900K						49%	1,13M	19s
36950K						49%	3,17M	19s
37000K						49%	1012K	19s
37050K						49%	968K	19s
37100K						50%	1,16M	19s
37150K						50%	2,48M	19s
37200K						50%	1,01M	19s
37250K						50%	1005K	19s
37300K						50%	976K	19s
37350K						50%	3,58M	19s
37400K						50%	1,12M	19s
37450K						50%	1,01M	19s
37500K						50%	1,14M	19s
37550K						50%	3,35M	19s
37600K						50%	948K	19s
37650K						50%	1010K	19s
37700K						50%	3,82M	19s
37750K						50%	1,12M	19s
37800K						50%	1007K	19s
37850K						51%	3,14M	19s
37900K						51%	1,18M	19s
37950K						51%	1,00M	19s
38000K						51%	1,04M	19s
38050K						51%	3,13M	19s
38100K						51%	1,06M	19s
38150K						51%	1,17M	19s
38200K						51%	2,69M	19s
38250K						51%	1,12M	19s
38300K						51%	2,44M	19s
38350K						51%	1,24M	19s
38400K						51%	955K	19s
38450K						51%	1,26M	19s
38500K						51%	2,67M	18s
38550K						51%	1,11M	18s
38600K						52%	4,48M	18s
38650K						52%	1012K	18s
38700K						52%	1,35M	18s
38750K						52%	2,98M	18s
38800K						52%	966K	18s

38850K						52%	3,80M	18s
38900K						52%	1,22M	18s
38950K						52%	1,27M	18s
39000K						52%	2,45M	18s
39050K						52%	1,16M	18s
39100K						52%	4,55M	18s
39150K						52%	1,03M	18s
39200K						52%	1,25M	18s
39250K						52%	2,53M	18s
39300K						52%	1,16M	18s
39350K						53%	4,44M	18s
39400K						53%	1,13M	18s
39450K						53%	2,92M	18s
39500K						53%	1,20M	18s
39550K						53%	1,17M	18s
39600K						53%	2,62M	18s
39650K						53%	1,28M	18s
39700K						53%	2,67M	18s
39750K						53%	1,32M	18s
39800K						53%	3,11M	18s
39850K						53%	1,23M	18s
39900K						53%	2,61M	18s
39950K						53%	1,21M	18s
40000K						53%	1,20M	18s
40050K						53%	2,72M	18s
40100K						54%	1,40M	18s
40150K						54%	2,52M	18s
40200K						54%	1,40M	18s
40250K						54%	2,44M	18s
40300K						54%	1,48M	18s
40350K						54%	2,50M	18s
40400K						54%	1,22M	18s
40450K						54%	2,30M	18s
40500K						54%	1,55M	18s
40550K						54%	2,85M	18s
40600K						54%	1,31M	18s
40650K						54%	3,11M	18s
40700K						54%	1,31M	17s
40750K						54%	3,16M	17s
40800K						54%	1,05M	17s
40850K						55%	1,38M	17s
40900K						55%	3,23M	17s
40950K						55%	3,92M	17s
41000K						55%	1,21M	17s
41050K						55%	3,92M	17s
41100K						55%	1,40M	17s
41150K						55%	3,44M	17s
41200K						55%	1,06M	17s
41250K						55%	4,62M	17s
41300K						55%	1,41M	17s
41350K						55%	2,70M	17s
41400K						55%	1,40M	17s
41450K						55%	3,48M	17s
41500K						55%	1,36M	17s
41550K						56%	2,88M	17s
41600K						56%	1,26M	17s

41650K						56%	2,89M	17s
41700K						56%	1,32M	17s
41750K						56%	3,21M	17s
41800K						56%	1,45M	17s
41850K						56%	2,84M	17s
41900K						56%	1,48M	17s
41950K						56%	2,77M	17s
42000K						56%	1,39M	17s
42050K						56%	2,69M	17s
42100K						56%	1,35M	17s
42150K						56%	3,53M	17s
42200K						56%	3,43M	17s
42250K						56%	1,36M	17s
42300K						57%	4,11M	17s
42350K						57%	1,30M	17s
42400K						57%	3,20M	17s
42450K						57%	1,31M	17s
42500K						57%	4,42M	17s
42550K						57%	1,25M	17s
42600K						57%	3,51M	16s
42650K						57%	3,33M	16s
42700K						57%	1,52M	16s
42750K						57%	3,79M	16s
42800K						57%	1,18M	16s
42850K						57%	3,28M	16s
42900K						57%	1,48M	16s
42950K						57%	2,82M	16s
43000K						57%	1,54M	16s
43050K						58%	3,29M	16s
43100K						58%	3,73M	16s
43150K						58%	1,43M	16s
43200K						58%	2,55M	16s
43250K						58%	1,45M	16s
43300K						58%	4,04M	16s
43350K						58%	1,40M	16s
43400K						58%	3,91M	16s
43450K						58%	3,04M	16s
43500K						58%	1,59M	16s
43550K						58%	3,15M	16s
43600K						58%	1,31M	16s
43650K						58%	4,12M	16s
43700K						58%	1,52M	16s
43750K						58%	3,19M	16s
43800K						59%	3,93M	16s
43850K						59%	1,44M	16s
43900K						59%	4,12M	16s
43950K						59%	1,51M	16s
44000K						59%	2,39M	16s
44050K						59%	1,50M	16s
44100K						59%	5,63M	16s
44150K						59%	769K	16s
44200K						59%	550M	16s
44250K						59%	6,80M	16s
44300K						59%	1,46M	16s
44350K						59%	2,17M	16s
44400K						59%	1,17M	15s

44450K						59%	1,35M	15s
44500K						59%	2,31M	15s
44550K						60%	1,32M	15s
44600K						60%	2,56M	15s
44650K						60%	1,45M	15s
44700K						60%	2,12M	15s
44750K						60%	1,63M	15s
44800K						60%	1,03M	15s
44850K						60%	3,11M	15s
44900K						60%	1,31M	15s
44950K						60%	3,04M	15s
45000K						60%	1,29M	15s
45050K						60%	3,24M	15s
45100K						60%	1,30M	15s
45150K						60%	3,15M	15s
45200K						60%	1,29M	15s
45250K						60%	2,36M	15s
45300K						61%	1,51M	15s
45350K						61%	3,35M	15s
45400K						61%	1,27M	15s
45450K						61%	3,36M	15s
45500K						61%	1,29M	15s
45550K						61%	3,28M	15s
45600K						61%	1,26M	15s
45650K						61%	3,38M	15s
45700K						61%	1,27M	15s
45750K						61%	3,39M	15s
45800K						61%	1,28M	15s
45850K						61%	4,22M	15s
45900K						61%	1,17M	15s
45950K						61%	5,50M	15s
46000K						61%	1,09M	15s
46050K						62%	4,44M	15s
46100K						62%	1,17M	15s
46150K						62%	9,54M	15s
46200K						62%	2,29M	15s
46250K						62%	1,81M	15s
46300K						62%	2,33M	15s
46350K						62%	1,51M	14s
46400K						62%	2,18M	14s
46450K						62%	1,58M	14s
46500K						62%	3,88M	14s
46550K						62%	1,19M	14s
46600K						62%	7,59M	14s
46650K						62%	1,05M	14s
46700K						62%	13,7M	14s
46750K						63%	2,76M	14s
46800K						63%	1,39M	14s
46850K						63%	2,75M	14s
46900K						63%	1,49M	14s
46950K						63%	4,40M	14s
47000K						63%	1,16M	14s
47050K						63%	8,88M	14s
47100K						63%	2,73M	14s
47150K						63%	1,60M	14s
47200K						63%	2,27M	14s

47250K						63%	1,56M	14s
47300K						63%	2,41M	14s
47350K						63%	1,52M	14s
47400K						63%	4,00M	14s
47450K						63%	1,21M	14s
47500K						64%	4,80M	14s
47550K						64%	4,38M	14s
47600K						64%	1,25M	14s
47650K						64%	3,59M	14s
47700K						64%	1,35M	14s
47750K						64%	4,89M	14s
47800K						64%	1,28M	14s
47850K						64%	4,63M	14s
47900K						64%	4,29M	14s
47950K						64%	1,28M	14s
48000K						64%	3,26M	14s
48050K						64%	1,41M	14s
48100K						64%	4,48M	14s
48150K						64%	1,28M	14s
48200K						64%	4,50M	13s
48250K						65%	4,39M	13s
48300K						65%	1,41M	13s
48350K						65%	2,67M	13s
48400K						65%	1,41M	13s
48450K						65%	4,42M	13s
48500K						65%	1,27M	13s
48550K						65%	4,34M	13s
48600K						65%	4,51M	13s
48650K						65%	1,43M	13s
48700K						65%	4,42M	13s
48750K						65%	1,27M	13s
48800K						65%	3,41M	13s
48850K						65%	5,97M	13s
48900K						65%	1,25M	13s
48950K						65%	3,72M	13s
49000K						66%	1,44M	13s
49050K						66%	4,38M	13s
49100K						66%	5,62M	13s
49150K						66%	1,25M	13s
49200K						66%	3,45M	13s
49250K						66%	1,44M	13s
49300K						66%	2,67M	13s
49350K						66%	1,60M	13s
49400K						66%	4,21M	13s
49450K						66%	4,24M	13s
49500K						66%	1,43M	13s
49550K						66%	2,63M	13s
49600K						66%	1,42M	13s
49650K						66%	5,03M	13s
49700K						66%	5,16M	13s
49750K						67%	1,20M	13s
49800K						67%	4,13M	13s
49850K						67%	1,42M	13s
49900K						67%	5,04M	13s
49950K						67%	5,24M	12s
50000K						67%	1,21M	12s

50050K						67%	3,92M	12s
50100K						67%	1,43M	12s
50150K						67%	2,63M	12s
50200K						67%	1,62M	12s
50250K						67%	4,53M	12s
50300K						67%	4,08M	12s
50350K						67%	1,40M	12s
50400K						67%	2,68M	12s
50450K						67%	1,62M	12s
50500K						68%	4,55M	12s
50550K						68%	3,81M	12s
50600K						68%	1,37M	12s
50650K						68%	5,87M	12s
50700K						68%	5,24M	12s
50750K						68%	606K	12s
50800K						68%	267M	12s
50850K						68%	5,02M	12s
50900K						68%	1,12M	12s
50950K						68%	5,27M	12s
51000K						68%	1,12M	12s
51050K						68%	2,18M	12s
51100K						68%	1,60M	12s
51150K						68%	2,17M	12s
51200K						68%	1,58M	12s
51250K						69%	961K	12s
51300K						69%	48,4M	12s
51350K						69%	2,11M	12s
51400K						69%	1,64M	12s
51450K						69%	2,06M	12s
51500K						69%	1,67M	12s
51550K						69%	2,11M	12s
51600K						69%	574K	12s
51650K						69%	429M	12s
51700K						69%	14,9M	12s
51750K						69%	999K	12s
51800K						69%	1,00M	12s
51850K						69%	7,05M	12s
51900K						69%	993K	12s
51950K						70%	1,00M	11s
52000K						70%	2,08M	11s
52050K						70%	1,44M	11s
52100K						70%	1,00M	11s
52150K						70%	7,93M	11s
52200K						70%	799K	11s
52250K						70%	1,22M	11s
52300K						70%	3,03M	11s
52350K						70%	1,14M	11s
52400K						70%	1008K	11s
52450K						70%	3,63M	11s
52500K						70%	1,05M	11s
52550K						70%	3,63M	11s
52600K						70%	1,12M	11s
52650K						70%	1,17M	11s
52700K						71%	3,61M	11s
52750K						71%	998K	11s
52800K						71%	1,16M	11s

52850K						71%	3,60M	11s
52900K						71%	995K	11s
52950K						71%	5,01M	11s
53000K						71%	1,13M	11s
53050K						71%	1,24M	11s
53100K						71%	3,09M	11s
53150K						71%	1,06M	11s
53200K						71%	1,16M	11s
53250K						71%	2,78M	11s
53300K						71%	1,14M	11s
53350K						71%	4,91M	11s
53400K						71%	1,06M	11s
53450K						72%	2,93M	11s
53500K						72%	1,20M	11s
53550K						72%	3,94M	11s
53600K						72%	1,13M	11s
53650K						72%	1,07M	11s
53700K						72%	2,84M	11s
53750K						72%	1,20M	11s
53800K						72%	4,05M	11s
53850K						72%	1,17M	11s
53900K						72%	3,24M	11s
53950K						72%	1,29M	11s
54000K						72%	1,11M	11s
54050K						72%	3,35M	11s
54100K						72%	1,17M	10s
54150K						72%	2,93M	10s
54200K						73%	1,29M	10s
54250K						73%	3,21M	10s
54300K						73%	1,12M	10s
54350K						73%	5,18M	10s
54400K						73%	1,10M	10s
54450K						73%	1,30M	10s
54500K						73%	3,18M	10s
54550K						73%	1,24M	10s
54600K						73%	3,54M	10s
54650K						73%	1,26M	10s
54700K						73%	3,43M	10s
54750K						73%	1,27M	10s
54800K						73%	2,56M	10s
54850K						73%	1,30M	10s
54900K						73%	3,13M	10s
54950K						74%	1,31M	10s
55000K						74%	3,15M	10s
55050K						74%	1,29M	10s
55100K						74%	3,85M	10s
55150K						74%	1,35M	10s
55200K						74%	2,08M	10s
55250K						74%	1,31M	10s
55300K						74%	3,10M	10s
55350K						74%	1,43M	10s
55400K						74%	3,45M	10s
55450K						74%	1,26M	10s
55500K						74%	4,04M	10s
55550K						74%	1,33M	10s
55600K						74%	2,70M	10s

55650K						74%	1,25M	10s
55700K						75%	4,08M	10s
55750K						75%	1,31M	10s
55800K						75%	3,10M	10s
55850K						75%	1,30M	10s
55900K						75%	3,09M	10s
55950K						75%	5,54M	10s
56000K						75%	1,11M	10s
56050K						75%	1,32M	9s
56100K						75%	3,10M	9s
56150K						75%	5,61M	9s
56200K						75%	1,41M	9s
56250K						75%	3,10M	9s
56300K						75%	1,32M	9s
56350K						75%	4,83M	9s
56400K						75%	1,15M	9s
56450K						76%	4,64M	9s
56500K						76%	1,33M	9s
56550K						76%	3,05M	9s
56600K						76%	1,33M	9s
56650K						76%	3,15M	9s
56700K						76%	5,12M	9s
56750K						76%	1,46M	9s
56800K						76%	2,53M	9s
56850K						76%	1,46M	9s
56900K						76%	2,83M	9s
56950K						76%	1,48M	9s
57000K						76%	3,49M	9s
57050K						76%	1,35M	9s
57100K						76%	3,10M	9s
57150K						77%	6,14M	9s
57200K						77%	1,09M	9s
57250K						77%	6,43M	9s
57300K						77%	1,41M	9s
57350K						77%	2,84M	9s
57400K						77%	1,54M	9s
57450K						77%	3,25M	9s
57500K						77%	7,29M	9s
57550K						77%	1,05M	9s
57600K						77%	7,16M	9s
57650K						77%	1,41M	9s
57700K						77%	2,71M	9s
57750K						77%	1,51M	9s
57800K						77%	3,32M	9s
57850K						77%	7,54M	9s
57900K						78%	1,06M	8s
57950K						78%	8,01M	8s
58000K						78%	1,41M	8s
58050K						78%	2,65M	8s
58100K						78%	1,51M	8s
58150K						78%	3,26M	8s
58200K						78%	7,76M	8s
58250K						78%	1,45M	8s
58300K						78%	2,61M	8s
58350K						78%	1,52M	8s
58400K						78%	3,06M	8s

58450K						78%	1,32M	8s
58500K						78%	3,12M	8s
58550K						78%	9,39M	8s
58600K						78%	1,50M	8s
58650K						79%	3,04M	8s
58700K						79%	10,2M	8s
58750K						79%	1,02M	8s
58800K						79%	9,62M	8s
58850K						79%	1,51M	8s
58900K						79%	3,06M	8s
58950K						79%	1,34M	8s
59000K						79%	3,00M	8s
59050K						79%	11,3M	8s
59100K						79%	1,48M	8s
59150K						79%	2,98M	8s
59200K						79%	1,35M	8s
59250K						79%	2,96M	8s
59300K						79%	10,9M	8s
59350K						79%	1,50M	8s
59400K						80%	2,67M	8s
59450K						80%	22,8M	8s
59500K						80%	1,50M	8s
59550K						80%	2,45M	8s
59600K						80%	1,48M	8s
59650K						80%	2,68M	8s
59700K						80%	23,2M	8s
59750K						80%	1,49M	7s
59800K						80%	2,66M	7s
59850K						80%	1,44M	7s
59900K						80%	2,66M	7s
59950K						80%	28,2M	7s
60000K						80%	1,45M	7s
60050K						80%	2,67M	7s
60100K						80%	27,4M	7s
60150K						81%	984K	7s
60200K						81%	22,4M	7s
60250K						81%	1,49M	7s
60300K						81%	2,56M	7s
60350K						81%	47,5M	7s
60400K						81%	1,37M	7s
60450K						81%	2,94M	7s
60500K						81%	44,5M	7s
60550K						81%	970K	7s
60600K						81%	52,3M	7s
60650K						81%	1,47M	7s
60700K						81%	2,24M	7s
60750K						81%	18,3M	7s
60800K						81%	1,59M	7s
60850K						81%	2,43M	7s
60900K						82%	18,8M	7s
60950K						82%	1,61M	7s
61000K						82%	2,27M	7s
61050K						82%	47,4M	7s
61100K						82%	1,03M	7s
61150K						82%	10,0M	7s
61200K						82%	597K	7s

61250K						82%	646M	7s
61300K						82%	959K	7s
61350K						82%	312M	7s
61400K						82%	2,15M	7s
61450K						82%	1,59M	7s
61500K						82%	2,19M	7s
61550K						82%	1,61M	7s
61600K						82%	2,12M	7s
61650K						83%	1,60M	6s
61700K						83%	2,23M	6s
61750K						83%	1,57M	6s
61800K						83%	2,15M	6s
61850K						83%	1,61M	6s
61900K						83%	2,23M	6s
61950K						83%	1,60M	6s
62000K						83%	2,16M	6s
62050K						83%	1,58M	6s
62100K						83%	719K	6s
62150K						83%	474M	6s
62200K						83%	7,65M	6s
62250K						83%	1,05M	6s
62300K						83%	1,37M	6s
62350K						84%	2,10M	6s
62400K						84%	1,03M	6s
62450K						84%	1,29M	6s
62500K						84%	2,37M	6s
62550K						84%	1,03M	6s
62600K						84%	1,39M	6s
62650K						84%	2,08M	6s
62700K						84%	1,51M	6s
62750K						84%	2,36M	6s
62800K						84%	945K	6s
62850K						84%	1,52M	6s
62900K						84%	2,34M	6s
62950K						84%	1,03M	6s
63000K						84%	1,40M	6s
63050K						84%	2,07M	6s
63100K						85%	1,50M	6s
63150K						85%	2,40M	6s
63200K						85%	1,02M	6s
63250K						85%	1,34M	6s
63300K						85%	2,26M	6s
63350K						85%	1,02M	6s
63400K						85%	9,34M	6s
63450K						85%	1,04M	6s
63500K						85%	1,33M	6s
63550K						85%	2,15M	6s
63600K						85%	1003K	6s
63650K						85%	4,16M	5s
63700K						85%	1,09M	5s
63750K						85%	1,49M	5s
63800K						85%	2,52M	5s
63850K						86%	1,43M	5s
63900K						86%	2,23M	5s
63950K						86%	1,33M	5s
64000K						86%	1,03M	5s

64050K						86%	2,74M	5s
64100K						86%	1,38M	5s
64150K						86%	2,78M	5s
64200K						86%	1,39M	5s
64250K						86%	2,74M	5s
64300K						86%	1,39M	5s
64350K						86%	2,30M	5s
64400K						86%	1005K	5s
64450K						86%	4,33M	5s
64500K						86%	1,17M	5s
64550K						86%	1,38M	5s
64600K						87%	2,76M	5s
64650K						87%	1,39M	5s
64700K						87%	2,77M	5s
64750K						87%	1,30M	5s
64800K						87%	2,69M	5s
64850K						87%	1,40M	5s
64900K						87%	2,74M	5s
64950K						87%	1,40M	5s
65000K						87%	2,70M	5s
65050K						87%	1,40M	5s
65100K						87%	2,10M	5s
65150K						87%	1,65M	5s
65200K						87%	2,04M	5s
65250K						87%	1,54M	5s
65300K						87%	2,34M	5s
65350K						88%	1,57M	5s
65400K						88%	2,22M	5s
65450K						88%	1,74M	5s
65500K						88%	1,97M	5s
65550K						88%	1,76M	5s
65600K						88%	1,90M	4s
65650K						88%	1,61M	4s
65700K						88%	2,19M	4s
65750K						88%	625K	4s
65800K						88%	366M	4s
65850K						88%	973K	4s
65900K						88%	10,4M	4s
65950K						88%	1,00M	4s
66000K						88%	948K	4s
66050K						88%	1,80M	4s
66100K						89%	1,82M	4s
66150K						89%	950K	4s
66200K						89%	1,84M	4s
66250K						89%	1,81M	4s
66300K						89%	1,88M	4s
66350K						89%	1,77M	4s
66400K						89%	951K	4s
66450K						89%	1,92M	4s
66500K						89%	1,74M	4s
66550K						89%	947K	4s
66600K						89%	22,2M	4s
66650K						89%	984K	4s
66700K						89%	1,86M	4s
66750K						89%	1,83M	4s
66800K						89%	952K	4s

66850K						90%	910K	4s
66900K						90%	418M	4s
66950K						90%	980K	4s
67000K						90%	1,81M	4s
67050K						90%	1,80M	4s
67100K						90%	948K	4s
67150K						90%	959K	4s
67200K						90%	933K	4s
67250K						90%	955K	4s
67300K						90%	52,3M	4s
67350K						90%	918K	4s
67400K						90%	953K	4s
67450K						90%	1,94M	4s
67500K						90%	1,70M	4s
67550K						91%	948K	4s
67600K						91%	947K	3s
67650K						91%	950K	3s
67700K						91%	1,93M	3s
67750K						91%	1,72M	3s
67800K						91%	948K	3s
67850K						91%	1,98M	3s
67900K						91%	1,70M	3s
67950K						91%	725K	3s
68000K						91%	941K	3s
68050K						91%	6,19M	3s
68100K						91%	1,07M	3s
68150K						91%	948K	3s
68200K						91%	6,03M	3s
68250K						91%	1,07M	3s
68300K						92%	944K	3s
68350K						92%	6,70M	3s
68400K						92%	945K	3s
68450K						92%	1,05M	3s
68500K						92%	950K	3s
68550K						92%	7,54M	3s
68600K						92%	1,03M	3s
68650K						92%	6,12M	3s
68700K						92%	1,08M	3s
68750K						92%	952K	3s
68800K						92%	5,81M	3s
68850K						92%	1,08M	3s
68900K						92%	951K	3s
68950K						92%	6,28M	3s
69000K						92%	1,07M	3s
69050K						93%	6,36M	3s
69100K						93%	1,05M	3s
69150K						93%	944K	3s
69200K						93%	6,45M	3s
69250K						93%	1,01M	3s
69300K						93%	1,01M	3s
69350K						93%	4,29M	3s
69400K						93%	1,11M	3s
69450K						93%	5,26M	3s
69500K						93%	1,09M	3s
69550K						93%	5,13M	2s
69600K						93%	986K	2s

69650K						93%	1,11M	2s
69700K						93%	5,64M	2s
69750K						93%	1,09M	2s
69800K						94%	4,50M	2s
69850K						94%	1,15M	2s
69900K						94%	4,71M	2s
69950K						94%	1,13M	2s
70000K						94%	938K	2s
70050K						94%	6,57M	2s
70100K						94%	1,06M	2s
70150K						94%	6,01M	2s
70200K						94%	1,08M	2s
70250K						94%	5,67M	2s
70300K						94%	1,09M	2s
70350K						94%	6,23M	2s
70400K						94%	967K	2s
70450K						94%	1,05M	2s
70500K						94%	7,56M	2s
70550K						95%	1,05M	2s
70600K						95%	7,49M	2s
70650K						95%	1,05M	2s
70700K						95%	7,86M	2s
70750K						95%	1,04M	2s
70800K						95%	7,55M	2s
70850K						95%	1,02M	2s
70900K						95%	9,16M	2s
70950K						95%	1,04M	2s
71000K						95%	9,50M	2s
71050K						95%	1,02M	2s
71100K						95%	9,39M	2s
71150K						95%	1,02M	2s
71200K						95%	8,42M	2s
71250K						95%	1,02M	2s
71300K						96%	9,39M	2s
71350K						96%	1,02M	2s
71400K						96%	9,50M	2s
71450K						96%	1,02M	1s
71500K						96%	12,4M	1s
71550K						96%	1018K	1s
71600K						96%	11,6M	1s
71650K						96%	1016K	1s
71700K						96%	14,4M	1s
71750K						96%	803K	1s
71800K						96%	46,7M	1s
71850K						96%	1006K	1s
71900K						96%	14,6M	1s
71950K						96%	488K	1s
72000K						96%	193M	1s
72050K						97%	1,09M	1s
72100K						97%	4,55M	1s
72150K						97%	998K	1s
72200K						97%	1,01M	1s
72250K						97%	5,33M	1s
72300K						97%	1,06M	1s
72350K						97%	6,55M	1s
72400K						97%	950K	1s

72450K		97%	1,00M	1s
72500K		97%	10,2M	1s
72550K		97%	954K	1s
72600K		97%	1,09M	1s
72650K		97%	5,82M	1s
72700K		97%	1,06M	1s
72750K		98%	6,89M	1s
72800K		98%	966K	1s
72850K		98%	498K	1s
72900K		98%	776M	1s
72950K		98%	9,28M	1s
73000K		98%	944K	1s
73050K		98%	1,03M	1s
73100K		98%	946K	1s
73150K		98%	9,28M	1s
73200K		98%	956K	1s
73250K		98%	952K	1s
73300K		98%	926K	1s
73350K		98%	947K	0s
73400K		98%	21,6M	0s
73450K		98%	955K	0s
73500K		99%	980K	0s
73550K		99%	985K	0s
73600K		99%	10,9M	0s
73650K		99%	946K	0s
73700K		99%	956K	0s
73750K		99%	972K	0s
73800K		99%	28,6M	0s
73850K		99%	959K	0s
73900K		99%	952K	0s
73950K		99%	44,9M	0s
74000K		99%	317K	0s
74050K		99%	203M	0s
74100K		99%	994K	0s
74150K		99%	15,9M	0s
74200K		99%	945K	0s
74250K		100%	642K=40s	

2025-02-25 13:17:52 (1,81 MB/s) - 'mot20.mp4' saved [76066403/76066403]

Класс VideoTracker

In [396...

```
class VideoTracker:
    def __init__(
        self,
        video_path: str,
        output_path: str,
        detector,
        tracker,
        cut_number_frames: int = 200,
        emb_size: int = 512,
        size_img: Tuple[int, int] = (128, 128),
        prefix: str = "Exp2",
        name_model: str = "convnext_tiny",
```

```

        path_to_weights: str = "./weights",
    ):
        """
        Инициализация трекера видео.

        :param video_path: Путь к входному видео.
        :param output_path: Путь для сохранения выходного видео.
        :param detector: Объект детектора (например, YoloDetector).
        :param tracker: Объект трекера (например, Tracker).
        :param cut_number_frames: Максимальное количество кадров для обработки.
        :param emb_size: Размер эмбеддингов.
        :param size_img: Размер изображений для модели.
        :param prefix: Префикс модели.
        :param name_model: Название модели.
        :param path_to_weights: Путь к весам модели.
        """
        self.video_path = video_path
        self.output_path = output_path
        self.detector = detector
        self.tracker = tracker
        self.cut_number_frames = cut_number_frames
        self.emb_size = emb_size
        self.size_img = size_img
        self.prefix = prefix
        self.name_model = name_model
        self.path_to_weights = path_to_weights

    def _get_embeddings(self, frame: np.ndarray, preds_bboxes: np.ndarray) -> np.ndarray:
        """
        Получение эмбеддингов для bounding boxes.

        :param frame: Входное изображение.
        :param preds_bboxes: Bounding boxes (N, 4).
        :return: Эмбеддинги (N, emb_size).
        """
        return get_embeddings(
            img=frame,
            preds_bboxes=preds_bboxes,
            emb_size=self.emb_size,
            size_img=self.size_img,
            prefix=self.prefix,
            name_model=self.name_model,
            path_to_weights=self.path_to_weights,
        )

    def _tracking_visualization(self, frame: np.ndarray, preds: np.ndarray) -> np.ndarray:
        """
        Визуализация трекинга на кадре.

        :param frame: Входное изображение.
        :param preds: Данные для визуализации (N, 5) – [x1, y1, x2, y2, track_id].
        :return: Изображение с визуализацией.
        """
        # Пример реализации (замените на вашу функцию визуализации)
        for pred in preds:
            x1, y1, x2, y2, track_id = map(int, pred)

```

```

        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(frame, str(track_id), (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPL
    return frame

def process_video(self):
    """
    Обработка видео: детекция, трекинг и визуализация.
    """
    try:
        cap = cv2.VideoCapture(self.video_path)
        if not cap.isOpened():
            raise ValueError("Ошибка при открытии видео")

        frame_size = (int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.C
        length = min(self.cut_number_frames, int(cap.get(cv2.CAP_PROP_FRAME_COU
        fps = cap.get(cv2.CAP_PROP_FPS)
        fourcc = cv2.VideoWriter_fourcc(*'avc1')
        video_writer = cv2.VideoWriter(self.output_path, fourcc, fps, frame_siz

        frame_count = 0
        pbar = tqdm(total=length)

        while True:
            ret, frame = cap.read()
            if not ret or frame_count > self.cut_number_frames:
                print("Стрим закончился")
                break

            # Конвертируем изображение в RGB
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

            # Детекция
            preds_masks, preds_bboxes = self.detector(frame)

            # Получение эмбедингов
            embeds = self._get_embeddings(frame, preds_bboxes)

            # Трекинг
            track_ids = self.tracker.tracking(frame_count=frame_count, masks=preds
            preds = np.column_stack([preds_bboxes, track_ids])

            # Визуализация
            out_frame = self._tracking_visualization(frame, preds)
            video_writer.write(cv2.cvtColor(out_frame, cv2.COLOR_RGB2BGR))

            frame_count += 1
            pbar.update(1)

        for path in [self.output_path]:
            os.system(f'ffmpeg -y -i "{path}" -c:v libx264 -pix_fmt yuv420p "{p

    finally:
        video_writer.release()
        cap.release()
        pbar.close()

```


t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 89.91image/s]

%|
| 25/200 [00:19<02:14, 1.30it/s]

13:33:00-035210 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 90.78image/s]

%|
26/200 [00:20<02:14, 1.30it/s]

13:33:00-788821 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 97.28image/s]

%|
| 27/200 [00:21<02:13, 1.30it/s]

13:33:01-544206 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 90.18image/s]

%|
| 28/200 [00:21<02:13, 1.29it/s]

13:33:02-304250 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 99.38image/s]

%|
| 29/200 [00:22<02:10, 1.31it/s]

13:33:03-044126 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 89.25image/s]

%|
| 30/200 [00:23<02:08, 1.32it/s]

13:33:03-787178 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/20 [00:00<?, ?image/s]

t Feature: 45%| 9/
20 [00:00<00:00, 89.80image/s]

Extract Feature: 100%|
| 20/20 [00:00<00:00, 76.04image/s]

%|
31/200 [00:24<02:10, 1.29it/s]

13:33:04-616726 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 89.84image/s]

%|
| 32/200 [00:24<02:09, 1.30it/s]

13:33:05-327716 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 91.32image/s]

%|
| 33/200 [00:25<02:05, 1.33it/s]

13:33:06-037137 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
18/18 [00:00<00:00, 103.42image/s]

%|
| 34/200 [00:26<02:01, 1.37it/s]

13:33:06-697273 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 99.99image/s]
```

```
%|
| 35/200 [00:26<01:57, 1.41it/s]
```

13:33:07-398494 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 93.34image/s]
```

```
%|
| 36/200 [00:27<01:56, 1.40it/s]
```

13:33:08-134643 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/19 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 19/19 [00:00<00:00, 94.32image/s]
```

```
%|
| 37/200 [00:28<01:57, 1.38it/s]
```

13:33:08-927318 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/19 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 19/19 [00:00<00:00, 92.53image/s]
```

```
%|
| 38/200 [00:29<02:00, 1.34it/s]
```

13:33:09-699963 INFO Модель convnext_tiny успешно загружена с весами из .

13:33:13-335438 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/19 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 19/19 [00:00<00:00, 93.90image/s]
```

```
%|
| 44/200 [00:33<01:53, 1.37it/s]
```

13:33:14-146294 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 83.94image/s]
```

```
%|
| 45/200 [00:34<01:56, 1.33it/s]
```

13:33:14-872794 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 95.73image/s]
```

```
%|
| 46/200 [00:35<01:54, 1.35it/s]
```

13:33:15-601145 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 92.15image/s]
```

```
%|
47/200 [00:35<01:53, 1.35it/s]
```

13:33:16-379410 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 91.42image/s]
```

```
%|
| 48/200 [00:36<01:54, 1.33it/s]
```

13:33:17-142306 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 93.39image/s]

%|
| 49/200 [00:37<01:54, 1.32it/s]

13:33:17-883092 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 90.46image/s]

%|
| 50/200 [00:38<01:52, 1.33it/s]

13:33:18-689950 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 91.58image/s]

%|
| 51/200 [00:38<01:54, 1.30it/s]

13:33:19-428489 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 96.98image/s]

%|
52/200 [00:39<01:52, 1.32it/s]

13:33:20-149122 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/20 [00:00<?, ?image/s]

t Feature: 50%| 10/2
0 [00:00<00:00, 95.33image/s]

Extract Feature: 100%|
| 20/20 [00:00<00:00, 95.73image/s]

%|
| 53/200 [00:40<01:50, 1.33it/s]

13:33:30-771092 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/17 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 17/17 [00:00<00:00, 99.85image/s]
```

```
%|
| 67/200 [00:51<01:37, 1.37it/s]
```

13:33:31-516577 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/16 [00:00<?, ?image/s]
```

```
t Feature: 44%| 7/1
6 [00:00<00:00, 69.83image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 78.68image/s]
```

```
%|
68/200 [00:51<01:38, 1.34it/s]
```

13:33:32-247419 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/17 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 17/17 [00:00<00:00, 88.00image/s]
```

```
%|
| 69/200 [00:52<01:36, 1.35it/s]
```

13:33:32-970374 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 95.67image/s]
```

```
%|
| 70/200 [00:53<01:34, 1.37it/s]
```

13:33:33-657635 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 87.13image/s]

%|
| 71/200 [00:53<01:32, 1.40it/s]

13:33:34-361820 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 91.07image/s]

%|
| 72/200 [00:54<01:31, 1.40it/s]

13:33:35-063639 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 99.44image/s]

%|
73/200 [00:55<01:30, 1.40it/s]

13:33:35-795998 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/21 [00:00<?, ?image/s]

Extract Feature: 100%|
| 21/21 [00:00<00:00, 95.18image/s]

%|
| 74/200 [00:56<01:32, 1.36it/s]

13:33:36-597155 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/21 [00:00<?, ?image/s]

Extract Feature: 100%|
21/21 [00:00<00:00, 102.01image/s]

%|
| 75/200 [00:56<01:33, 1.34it/s]

13:33:37-337219 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 94.67image/s]

%|
| 76/200 [00:57<01:31, 1.35it/s]

13:33:38-083758 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

t Feature: 42%| 8/1
9 [00:00<00:00, 75.84image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 78.00image/s]

%|
| 77/200 [00:58<01:33, 1.32it/s]

13:33:38-886357 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
17/17 [00:00<00:00, 100.34image/s]

%|
78/200 [00:59<01:31, 1.34it/s]

13:33:39-559335 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 93.04image/s]

%|
| 79/200 [00:59<01:28, 1.36it/s]

13:33:40-283373 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 94.54image/s]

%|
| 80/200 [01:00<01:27, 1.37it/s]

13:33:40-970288 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 93.93image/s]
```

```
%|
| 81/200 [01:01<01:24, 1.41it/s]
```

13:33:41-685432 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/14 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 14/14 [00:00<00:00, 96.45image/s]
```

```
%|
| 82/200 [01:01<01:22, 1.42it/s]
```

13:33:42-384396 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/17 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 17/17 [00:00<00:00, 95.14image/s]
```

```
%|
83/200 [01:02<01:23, 1.40it/s]
```

13:33:43-120831 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/15 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 15/15 [00:00<00:00, 85.00image/s]
```

```
%|
84/200 [01:03<01:23, 1.39it/s]
```

13:33:43-857600 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
16/16 [00:00<00:00, 103.07image/s]
```

```
%|
| 85/200 [01:04<01:22, 1.39it/s]
```

13:33:44-573583 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 68.90image/s]

%|
| 86/200 [01:04<01:23, 1.36it/s]

13:33:45-335180 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 99.46image/s]

%|
| 87/200 [01:05<01:21, 1.38it/s]

13:33:46-008218 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 98.87image/s]

%|
| 88/200 [01:06<01:19, 1.41it/s]

13:33:46-687585 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 97.42image/s]

%|
89/200 [01:06<01:17, 1.43it/s]

13:33:47-370246 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
15/15 [00:00<00:00, 101.00image/s]

%|
| 90/200 [01:07<01:16, 1.44it/s]

13:33:48-073937 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 84.45image/s]

%|
| 91/200 [01:08<01:16, 1.42it/s]

13:33:48-854781 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 86.89image/s]

%|
| 92/200 [01:09<01:18, 1.38it/s]

13:33:49-598773 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 88.66image/s]

%|
| 93/200 [01:09<01:18, 1.36it/s]

13:33:50-343991 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 89.30image/s]

%|
94/200 [01:10<01:18, 1.36it/s]

13:33:51-077034 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 85.35image/s]

%|
| 95/200 [01:11<01:18, 1.35it/s]

13:33:51-869421 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
13:33:52-613303 INFO    Модель convnext_tiny успешно загружена с весами из .
```

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
13:33:53-384677 INFO    Модель convnext_tiny успешно загружена с весами из .
```

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
13:33:54-142891 INFO    Модель convnext_tiny успешно загружена с весами из .
```

```
t Feature: 0%|
| 0/17 [00:00<?, ?image/s]
```

```
13:33:54-890323 INFO    Модель convnext_tiny успешно загружена с весами из .
```

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 96.65image/s]

%|
100/200 [01:15<01:14, 1.34it/s]

13:33:55-640892 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
17/17 [00:00<00:00, 101.41image/s]

%|
101/200 [01:15<01:13, 1.34it/s]

13:33:56-364668 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

t Feature: 50%|
8 [00:00<00:00, 89.51image/s]

| 9/1

Extract Feature: 100%|
| 18/18 [00:00<00:00, 88.55image/s]

%|
102/200 [01:16<01:13, 1.34it/s]

13:33:57-107116 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 96.91image/s]

%|
103/200 [01:17<01:11, 1.35it/s]

13:33:57-794705 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
17/17 [00:00<00:00, 102.22image/s]

%|
104/200 [01:18<01:09, 1.39it/s]

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 93.87image/s]

%|
109/200 [01:21<01:03, 1.42it/s]

13:34:02-073984 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 90.52image/s]

%|
110/200 [01:22<01:03, 1.42it/s]

13:34:02-781974 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 94.43image/s]

%|
111/200 [01:23<01:03, 1.41it/s]

13:34:03-461907 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 88.05image/s]

%|
112/200 [01:23<01:01, 1.43it/s]

13:34:04-165227 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 94.51image/s]

%|
113/200 [01:24<01:00, 1.43it/s]

13:34:04-853265 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 94.87image/s]

%|
114/200 [01:25<00:59, 1.44it/s]

13:34:05-587184 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 62.58image/s]

%|
115/200 [01:25<01:02, 1.37it/s]

13:34:06-351476 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 92.18image/s]

%|
116/200 [01:26<01:00, 1.38it/s]

13:34:07-075039 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 94.29image/s]

%|
117/200 [01:27<00:59, 1.39it/s]

13:34:07-786388 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 85.46image/s]

%|
118/200 [01:28<00:59, 1.37it/s]

13:34:08-562105 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 95.11image/s]

%|
119/200 [01:28<00:59, 1.36it/s]

13:34:09-277297 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
19/19 [00:00<00:00, 100.62image/s]

%|
120/200 [01:29<00:58, 1.37it/s]

13:34:10-005187 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/20 [00:00<?, ?image/s]

Extract Feature: 100%|
20/20 [00:00<00:00, 101.38image/s]

%|
121/200 [01:30<00:57, 1.36it/s]

13:34:10-752341 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 98.47image/s]

%|
122/200 [01:31<00:57, 1.36it/s]

13:34:11-507053 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 90.58image/s]

%|
123/200 [01:31<00:57, 1.35it/s]

13:34:12-255392 INFO Модель convnext_tiny успешно загружена с весами из .

13:34:15-886100 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/17 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 17/17 [00:00<00:00, 84.88image/s]
```

```
%|
129/200 [01:36<00:51, 1.37it/s]
```

13:34:16-632648 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/15 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 15/15 [00:00<00:00, 100.53image/s]
```

```
%|
130/200 [01:36<00:50, 1.39it/s]
```

13:34:17-333660 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/15 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 15/15 [00:00<00:00, 96.28image/s]
```

```
%|
131/200 [01:37<00:49, 1.40it/s]
```

13:34:18-059459 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 95.98image/s]
```

```
%|
132/200 [01:38<00:49, 1.39it/s]
```

13:34:18-765346 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 79.95image/s]
```

```
%|
133/200 [01:39<00:48, 1.37it/s]
```

13:34:19-515930 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 95.19image/s]

%|
134/200 [01:39<00:47, 1.39it/s]

13:34:20-195998 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/13 [00:00<?, ?image/s]

Extract Feature: 100%|
13/13 [00:00<00:00, 101.96image/s]

%|
135/200 [01:40<00:45, 1.43it/s]

13:34:20-825713 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/14 [00:00<?, ?image/s]

Extract Feature: 100%|
| 14/14 [00:00<00:00, 96.25image/s]

%|
136/200 [01:41<00:43, 1.46it/s]

13:34:21-481316 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/12 [00:00<?, ?image/s]

Extract Feature: 100%|
| 12/12 [00:00<00:00, 90.93image/s]

%|
137/200 [01:41<00:42, 1.49it/s]

13:34:22-145213 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/14 [00:00<?, ?image/s]

Extract Feature: 100%|
| 14/14 [00:00<00:00, 94.21image/s]

%|
138/200 [01:42<00:41, 1.48it/s]

13:34:22-827553 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/11 [00:00<?, ?image/s]

Extract Feature: 100%|
| 11/11 [00:00<00:00, 96.96image/s]

%|
139/200 [01:43<00:40, 1.50it/s]

13:34:23-473563 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/12 [00:00<?, ?image/s]

Extract Feature: 100%|
| 12/12 [00:00<00:00, 96.83image/s]

%|
140/200 [01:43<00:39, 1.51it/s]

13:34:24-141049 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/13 [00:00<?, ?image/s]

Extract Feature: 100%|
| 13/13 [00:00<00:00, 91.28image/s]

%|
141/200 [01:44<00:39, 1.49it/s]

13:34:24-801863 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/14 [00:00<?, ?image/s]

t Feature: 43%|
4 [00:00<00:00, 58.66image/s]

| 6/1

Extract Feature: 100%|
| 14/14 [00:00<00:00, 55.61image/s]

%|
142/200 [01:45<00:40, 1.43it/s]

13:34:25-568425 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
15/15 [00:00<00:00, 102.37image/s]

%|
143/200 [01:45<00:39, 1.45it/s]

13:34:26-226655 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 93.85image/s]
```

```
%|
144/200 [01:46<00:38, 1.46it/s]
```

13:34:26-923315 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 99.28image/s]
```

```
%|
145/200 [01:47<00:38, 1.44it/s]
```

13:34:27-628204 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 98.70image/s]
```

```
%|
146/200 [01:47<00:37, 1.43it/s]
```

13:34:28-361825 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/19 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 19/19 [00:00<00:00, 95.01image/s]
```

```
%|
147/200 [01:48<00:37, 1.40it/s]
```

13:34:29-112184 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/20 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 20/20 [00:00<00:00, 96.74image/s]
```

```
%|
148/200 [01:49<00:37, 1.38it/s]
```

13:34:29-889009 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 96.10image/s]

%|
149/200 [01:50<00:37, 1.36it/s]

13:34:30-639194 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 94.74image/s]

%|
150/200 [01:50<00:36, 1.35it/s]

13:34:31-384214 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 94.50image/s]

%|
151/200 [01:51<00:36, 1.34it/s]

13:34:32-131081 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

t Feature: 39%|
8 [00:00<00:00, 68.26image/s]

| 7/1

Extract Feature: 100%|
| 18/18 [00:00<00:00, 81.66image/s]

%|
152/200 [01:52<00:36, 1.33it/s]

13:34:32-926635 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 94.79image/s]

%|
153/200 [01:53<00:35, 1.32it/s]

13:34:33-642556 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 97.50image/s]
```

```
%|
154/200 [01:53<00:34, 1.34it/s]
```

13:34:34-393292 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/15 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 15/15 [00:00<00:00, 89.39image/s]
```

```
%|
155/200 [01:54<00:33, 1.36it/s]
```

13:34:35-097676 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/17 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 17/17 [00:00<00:00, 103.33image/s]
```

```
%|
156/200 [01:55<00:32, 1.37it/s]
```

13:34:35-831587 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/17 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 17/17 [00:00<00:00, 86.49image/s]
```

```
%|
157/200 [01:56<00:31, 1.35it/s]
```

13:34:36-568563 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature: 0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 86.06image/s]
```

```
%|
158/200 [01:56<00:30, 1.36it/s]
```

13:34:37-278361 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 99.90image/s]

%|
159/200 [01:57<00:29, 1.39it/s]

13:34:37-978667 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 96.45image/s]

%|
160/200 [01:58<00:28, 1.39it/s]

13:34:38-739142 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

t Feature: 44%| 8/1
8 [00:00<00:00, 74.34image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 78.93image/s]

%|
161/200 [01:59<00:29, 1.34it/s]

13:34:39-497559 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 94.70image/s]

%|
162/200 [01:59<00:27, 1.36it/s]

13:34:40-190882 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 98.05image/s]

%|
163/200 [02:00<00:26, 1.38it/s]

13:34:40-922981 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/18 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 18/18 [00:00<00:00, 90.84image/s]
```

```
%|
164/200 [02:01<00:26, 1.37it/s]
```

13:34:41-655617 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/16 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 16/16 [00:00<00:00, 88.03image/s]
```

```
%|
165/200 [02:01<00:25, 1.38it/s]
```

13:34:42-388301 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/17 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 17/17 [00:00<00:00, 97.95image/s]
```

```
%|
166/200 [02:02<00:24, 1.38it/s]
```

13:34:43-089546 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/19 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
| 19/19 [00:00<00:00, 93.70image/s]
```

```
%|
167/200 [02:03<00:24, 1.37it/s]
```

13:34:43-835184 INFO Модель convnext_tiny успешно загружена с весами из .

```
t Feature:  0%|
| 0/19 [00:00<?, ?image/s]
```

```
Extract Feature: 100%|
19/19 [00:00<00:00, 103.04image/s]
```

```
%|
168/200 [02:04<00:23, 1.37it/s]
```

13:34:44-571793 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 87.02image/s]

%|
173/200 [02:08<00:20, 1.31it/s]

13:34:48-476748 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 92.27image/s]

%|
174/200 [02:08<00:19, 1.33it/s]

13:34:49-204814 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 96.98image/s]

%|
175/200 [02:09<00:18, 1.35it/s]

13:34:49-939202 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 95.40image/s]

%|
176/200 [02:10<00:17, 1.35it/s]

13:34:50-687199 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 94.57image/s]

%|
177/200 [02:10<00:17, 1.35it/s]

13:34:51-422619 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 89.79image/s]

%|
182/200 [02:14<00:13, 1.32it/s]

13:34:55-258573 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 92.54image/s]

%|
183/200 [02:15<00:12, 1.33it/s]

13:34:56-030458 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 94.18image/s]

%|
184/200 [02:16<00:12, 1.32it/s]

13:34:56-772059 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 90.89image/s]

%|
185/200 [02:17<00:11, 1.33it/s]

13:34:57-515843 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 87.24image/s]

%|
186/200 [02:17<00:10, 1.32it/s]

13:34:58-299028 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/18 [00:00<?, ?image/s]

Extract Feature: 100%|
| 18/18 [00:00<00:00, 93.17image/s]

%|
187/200 [02:18<00:09, 1.31it/s]

13:34:59-049177 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 88.27image/s]

%|
188/200 [02:19<00:09, 1.32it/s]

13:34:59-784489 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 97.28image/s]

%|
189/200 [02:20<00:08, 1.35it/s]

13:35:00-503712 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

t Feature: 47%| 9/
19 [00:00<00:00, 81.48image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 80.36image/s]

%|
190/200 [02:20<00:07, 1.32it/s]

13:35:01-287322 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/19 [00:00<?, ?image/s]

Extract Feature: 100%|
| 19/19 [00:00<00:00, 97.62image/s]

%|
191/200 [02:21<00:06, 1.33it/s]

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 92.73image/s]

%|
196/200 [02:25<00:03, 1.30it/s]

13:35:06-127025 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 94.71image/s]

%|
197/200 [02:26<00:02, 1.33it/s]

13:35:06-846900 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/17 [00:00<?, ?image/s]

Extract Feature: 100%|
| 17/17 [00:00<00:00, 96.03image/s]

%|
198/200 [02:27<00:01, 1.34it/s]

13:35:07-564729 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/16 [00:00<?, ?image/s]

Extract Feature: 100%|
| 16/16 [00:00<00:00, 94.33image/s]

%|
199/200 [02:27<00:00, 1.36it/s]

13:35:08-304883 INFO Модель convnext_tiny успешно загружена с весами из .

t Feature: 0%|
| 0/15 [00:00<?, ?image/s]

Extract Feature: 100%|
| 15/15 [00:00<00:00, 87.97image/s]

%|
200/200 [02:28<00:00, 1.36it/s]

13:35:09-015819 INFO Модель convnext_tiny успешно загружена с весами из .

Выводы

В целом все неплохо работает, всем советую
